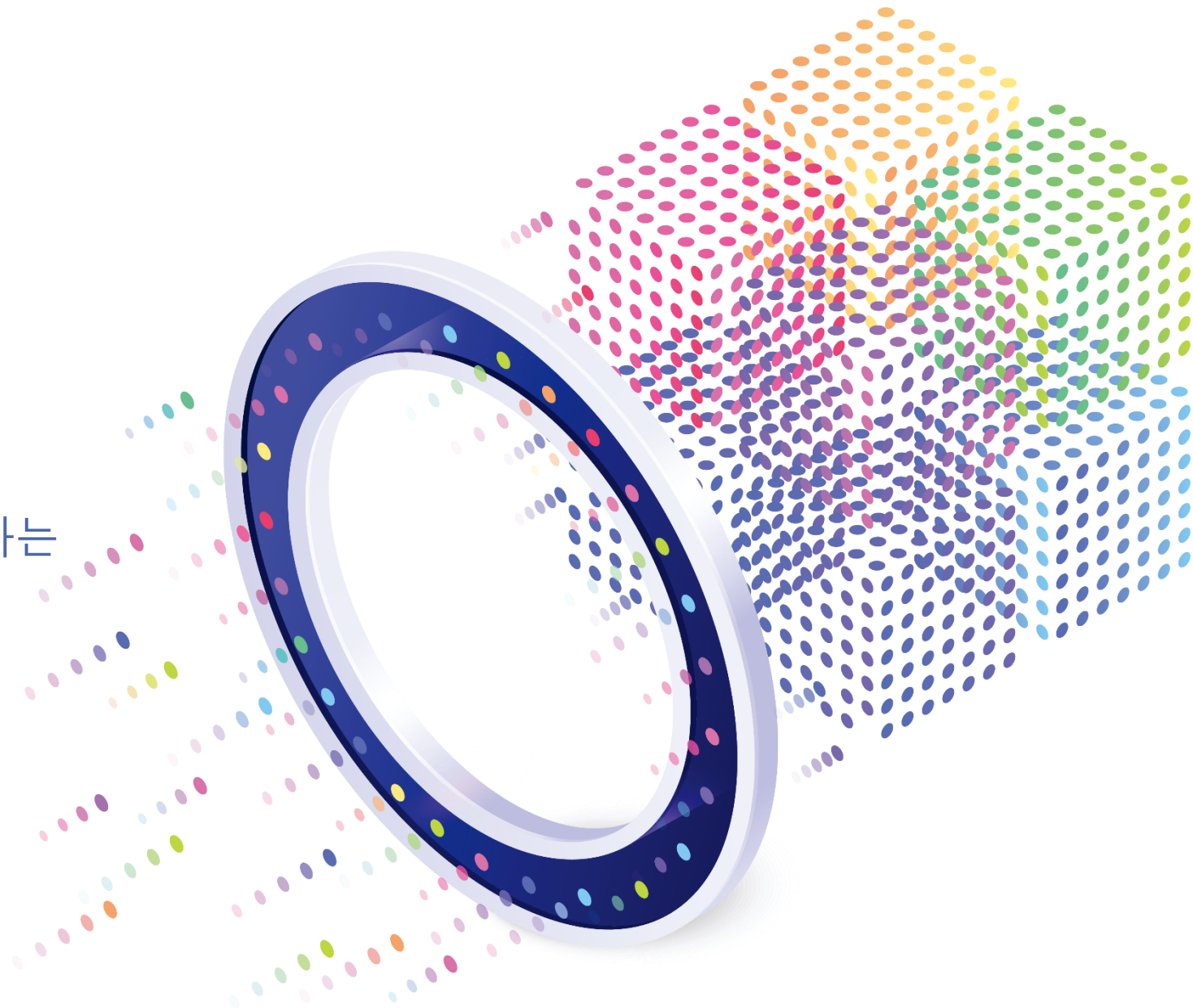


Azure Event Hubs

네이티브 Apache Kafka 지원을 사용하는
실시간 데이터 스트리밍 플랫폼



이 자료는 Elixir의 사전 서면 승인 없이 외부에 배포하기 위해 그 일부를 배포, 인용 또는 복제할 수 없습니다.

© Copyright Elixir

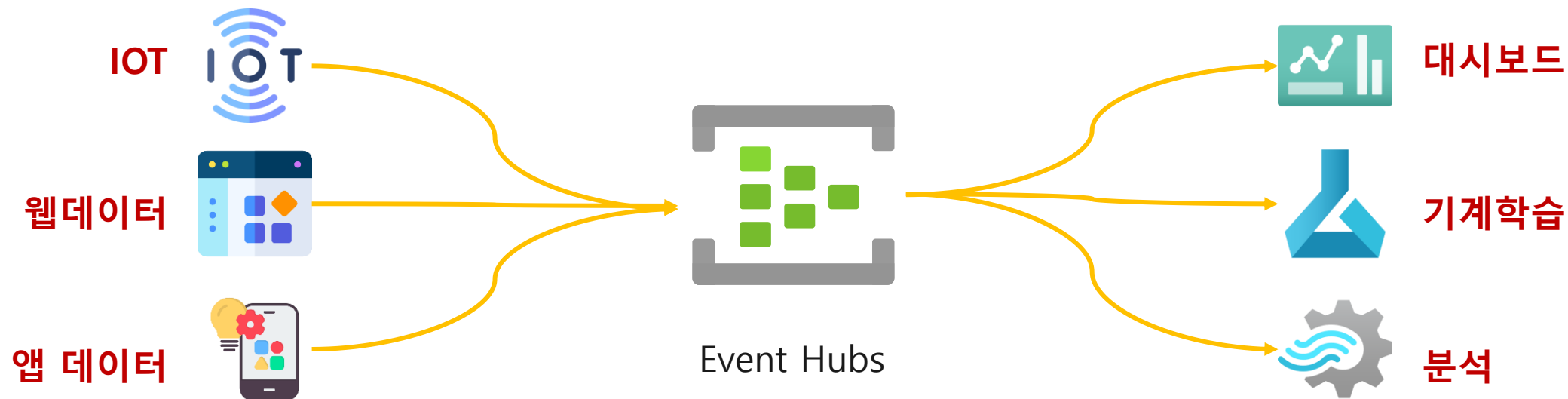
Azure Event Hubs

네이티브 Apache Kafka 지원을 사용하는
실시간 데이터 스트리밍 플랫폼

Azure Event Hubs 란 무엇일까요?

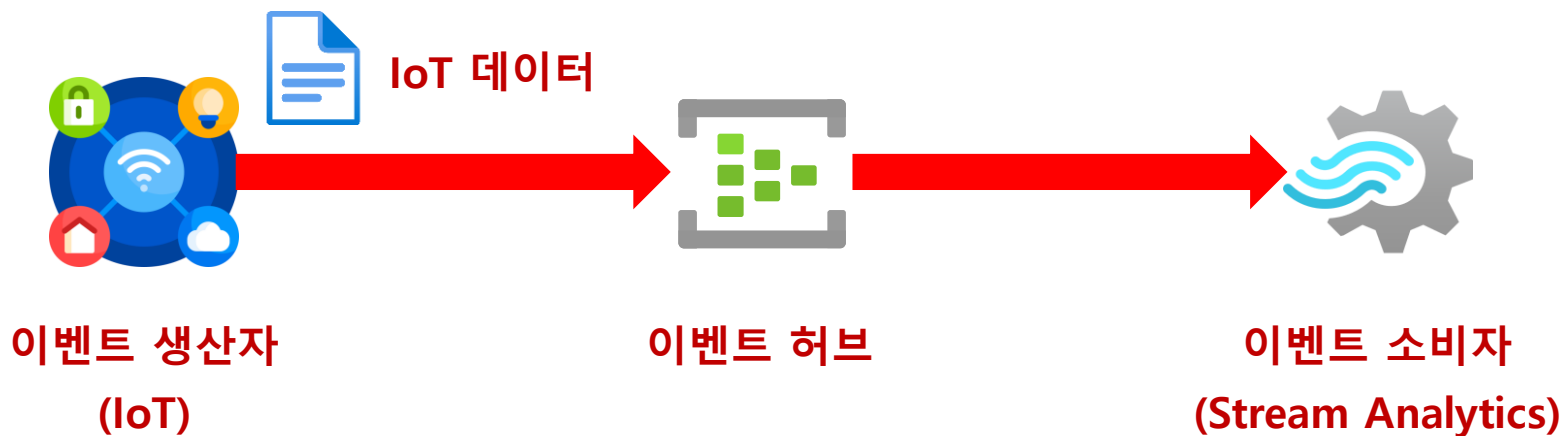
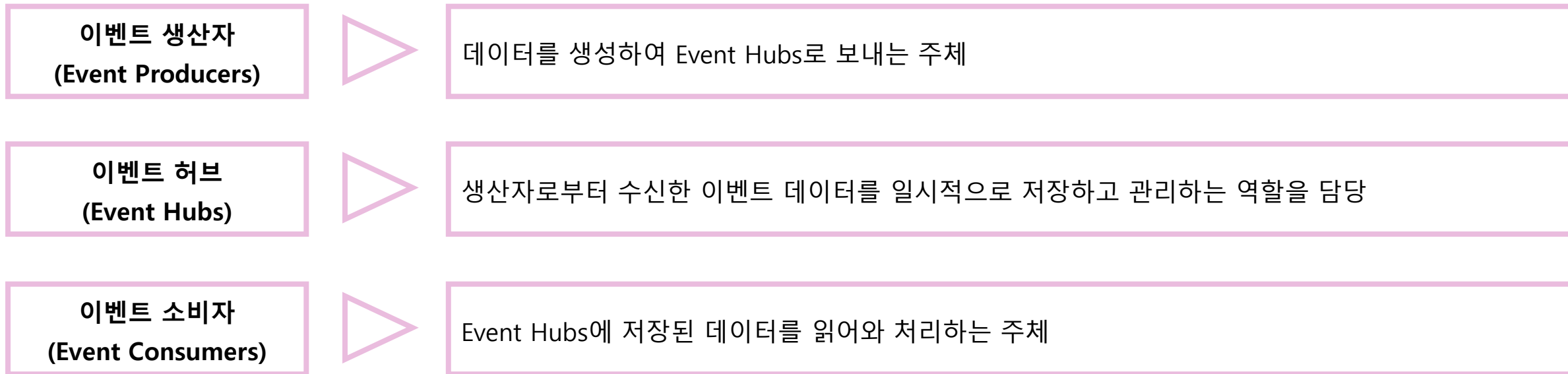
Azure Event Hubs는 확장성이 뛰어난 데이터 스트리밍 플랫폼이자 이벤트 수집 서비스입니다.

- 실시간으로 대량의 데이터를 안정적이고 신뢰성 있게 수집, 처리 및 저장
- IoT 센서, 모바일 앱, 웹사이트 클릭 스트림 등 다양한 소스에서 발생하는 이벤트 처리
- 수집된 데이터를 실시간 분석, 대시보드 구축, 트랜잭션 처리 등 다양한 목적으로 활용
- Event Hubs의 주요 특징: 대규모 병렬 처리, 쉬운 연동 (Kafka 프로토콜 지원 등)



Azure Event Hubs 아키텍처 이해하기

Azure Event Hubs의 아키텍처는 크게 세 가지 주요 구성 요소로 이루어져 있습니다.



Event Hubs 핵심 기능: Kafka 호환

Kafka는 LinkedIn에서 개발되어 현재는 Apache 재단에서 관리하는 오픈 소스 스트리밍 플랫폼입니다.



Kafka 특징:

- 대용량 실시간 데이터 피드를 효율적으로 관리할 수 있음
(예를 들어, 데이터를 안정적으로 대량 전송해야 하는 환경에서 널리 사용됨)
- 실시간 데이터 처리 및 분석 파이프라인을 구축할 수 있음
(전 세계 많은 기업들이 Kafka를 활용하고 있음)

Event Hubs의 Kafka 호환성:

- Event Hubs는 Kafka 프로토콜을 완벽하게 지원
- 기존 Kafka 애플리케이션을 코드 변경 없이 그대로 Event Hubs에서 실행할 수 있음

Kafka에서 Event Hubs로의 마이그레이션:

- 강력한 Kafka 호환성: 기존 Kafka 사용자는 쉽고 빠르게 Event Hubs로 전환할 수 있음.
- Azure에서 제공하는 PaaS 형태의 Event Hubs : 직접 Kafka 클러스터를 설치하고 관리할 필요 없어짐
- 마이그레이션 효과: 인프라 관리 부담을 줄이고 비용을 절감할 수 있음

Event Hubs 핵심 기능: 스키마 레지스트리(1)

스키마란 무엇일까요?

- 스키마는 데이터의 구조와 형식을 정의하는 청사진입니다.
- JSON 데이터로 예를 들면, 객체의 속성과 각 속성의 데이터 타입을 명확하게 정의하는 것

생산자와 소비자의 스키마 계약 역할:

- 생산자: 스키마에 맞춰 데이터를 생성
- 소비자: 스키마를 기반으로 데이터를 올바르게 해석하고 처리

스키마 레지스트리가 왜 필요할까요?

- 다양한 생산자와 소비자 구성
- 스키마 레지스트리가 필요한 이유: 생산자와 소비자들이 모두 동일한 스키마를 사용한다고 보장할 수 없음

스키마 레지스트리가 없을 때 발생할 수 있는 오류:

- 데이터 처리 오류가 발생: 생산자가 스키마를 변경했는데 소비자가 이를 모름
- 데이터 호환성 오류가 발생: 생산자와 소비자가 서로 다른 스키마를 사용함

```
{  
  "type": "record",  
  "name": "SensorReading",  
  "fields": [  
    {"name": "deviceId", "type": "string"},  
    {"name": "temperature", "type": "float"},  
    {"name": "humidity", "type": "float"},  
    {"name": "timestamp", "type": "long"}  
  ]  
}
```

Event Hubs 핵심 기능: 스키마 레지스트리(2)

Azure Schema Registry의 특징

- 중앙 집중식 스키마 저장소
- Event Hubs와 통합되어 사용할 수 있음

주요 기능은 다음과 같습니다:



1. 스키마 저장 및 버전 관리

- 스키마를 중앙 레지스트리에 저장하고, 버전별로 관리
- 스키마 진화(Schema Evolution)를 추적할 수 있어, 데이터 호환성을 보장



2. 스키마 검증

- 생산자가 보내는 데이터가 등록된 스키마와 일치하는지 자동으로 검증
- 데이터 품질을 보장: 스키마와 일치하지 않는 데이터는 거부됨



3. 스키마 조회

- 소비자는 레지스트리에서 데이터의 스키마를 조회 가능
- 소비자는 항상 최신 스키마를 사용하여 데이터를 처리할 수 있음

Azure Event Hubs 활용 분야 살펴보기(1)

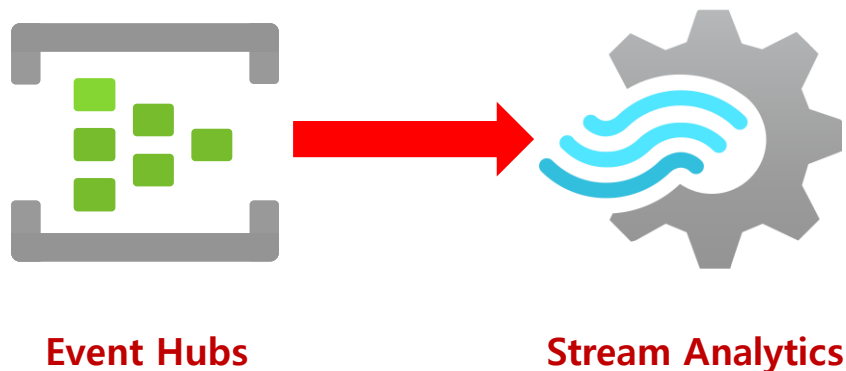
Azure Event Hubs는 다양한 분야에서 활용되는 강력한 실시간 데이터 스트리밍 플랫폼입니다.

실시간 데이터 분석의 중요성:

- 과거 데이터 분석도 중요함
- 빠르게 변화하는 현재 상황에 대응하기 위해서는 실시간 데이터 분석이 필수적임
- 현재 발생하는 이벤트를 즉시 분석하고 대응함으로써 기회를 놓치지 않고 문제에 신속하게 대처할 수 있음

Event Hubs와 Stream Analytics: 완벽한 실시간 분석 파트너

Azure Event Hubs와 Azure Stream Analytics를 함께 사용하면 강력한 실시간 분석 파이프라인을 구축할 수 있습니다.



Azure Event Hubs 활용 분야 살펴보기(2)

Event Hubs 활용 사례



실시간 대시보드

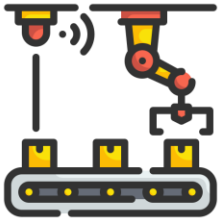


**핵 사용 감지
(온라인 게임)**

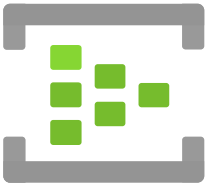
실시간 대시보드 (생산 라인, IoT 센서)	• 데이터 실시간으로 모니터링 • 데이터 시각화
이상 감지 (금융 거래, 보안 시스템 등)	• 이상 징후를 실시간으로 감지 • 이상 징후 즉각 알림
개인 맞춤형 서비스	• 사용자 행동을 실시간 분석 • 개인화된 추천 서비스 제공
핵 사용 감지 (온라인 게임)	• 핵 사용 유저를 즉시 감지 • 핵 사용 유저 제재 가능

실전 시나리오: Event Hubs를 활용한 실시간 데이터 수집

스마트 팩토리 환경에서 IoT 센서 데이터를 실시간으로 수집하고 분석하는 시나리오



1. **스마트 팩토리 환경**: 설치된 수많은 IoT 센서가 생산 설비의 상태, 환경 데이터 등 실시간 생성



2. **Event Hubs로 데이터 수집**: 센서 데이터를 MQTT, AMQP 등 프로토콜을 통해 Event Hubs로 전송



3. **Azure Functions로 실시간 분석**: Event Hubs 트리거를 사용하여 Azure Functions를 실행



4. **분석 결과 활용**: 실시간 분석된 결과는 다양한 용도로 활용될 수 있습니다.

- 생산 라인 모니터링 대시보드
- 이상 징후 감지 알림
- 생산 효율성 개선

Azure Portal을 사용하여 Event Hub 생성

Azure Event Hubs, Azure Portal로 간편하게 시작하기

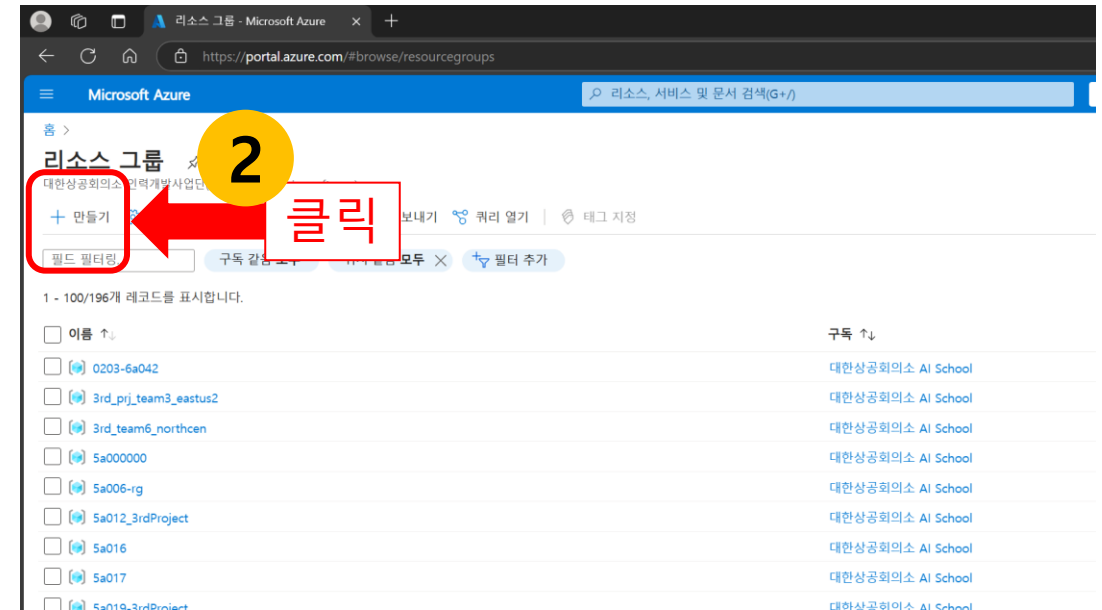
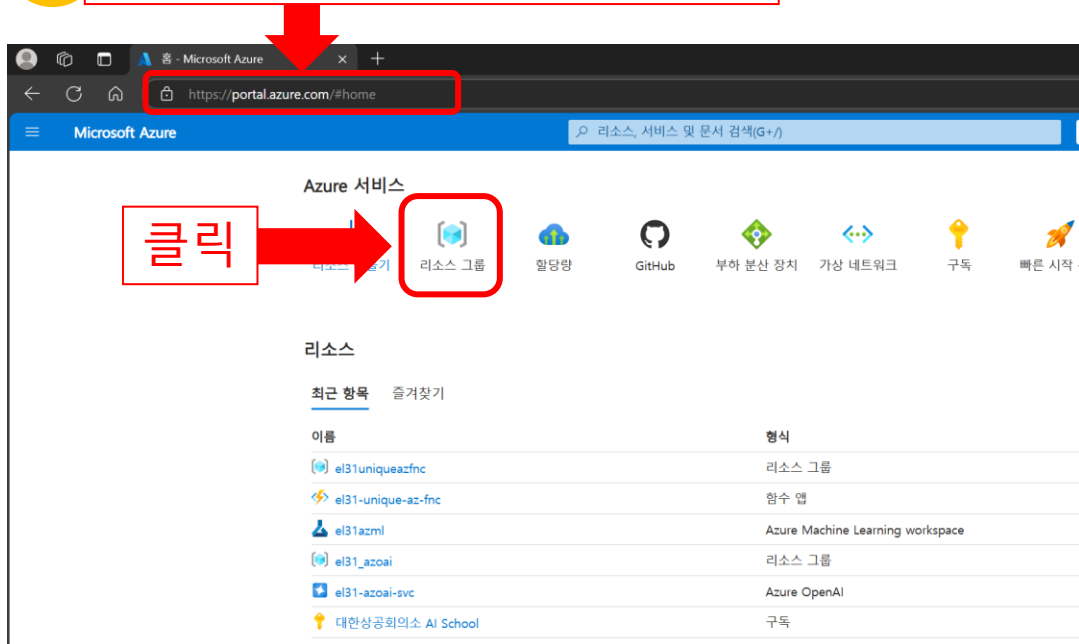
리소스 그룹 생성 (1)

Event Hubs를 사용하려면 먼저 리소스 그룹을 생성해야 합니다.

리소스 그룹, 생성 방법:

1. Azure Portal에 로그인합니다. (<https://portal.azure.com>)
2. '리소스 그룹' 메뉴를 선택 한 뒤 '+ 만들기' 버튼을 클릭합니다.

1 <https://portal.azure.com/> 접속



리소스 그룹 생성 (2)

3. 리소스 그룹을 만들 'Azure 구독'을 선택합니다. (예: 대한상공회의소 AI School)
4. 리소스 그룹 이름은 고유한 이름을 입력합니다. (예: el31-solar-hubs-rg)
5. 원하는 Azure 지역을 선택합니다. (예: (US) East US)
6. '검토 + 만들기' 버튼을 클릭 후 '만들기' 버튼을 클릭합니다.

Microsoft Azure

리소스, 서비스 및 문서 검색(G+)

리소스 그룹 만들기

기본

리소스 그룹 - Azure 솔루션의 관련 리소스를 보관하는 컨테이너입니다. 리소스 그룹에 솔루션의 모든 리소스를 포함할 수도 있고 그룹으로 관리할 리소스만 포함할 수도 있습니다. 무엇이 조직에 가장 적합한지에 따라 리소스 그룹에 리소스를 할당할 방법을 결정합니다. 자세한 정보 >

프로젝트 정보

구독 * ○

리소스 그룹 * ○

리소스 세부 정보

영역 * ○

대한상공회의소 AI School

el31-solar-hubs-rg

(US) East US

검토 + 만들기

만들기

Microsoft Azure

리소스, 서비스 및 문서 검색(G+)

리소스 그룹 만들기

유효성 검사를 통과했습니다.

기본 태그 검토 + 만들기

기본

구독 대한상공회의소 AI School

리소스 그룹 el31-solar-hubs-rg

영역 East US

태그

없음

만들기

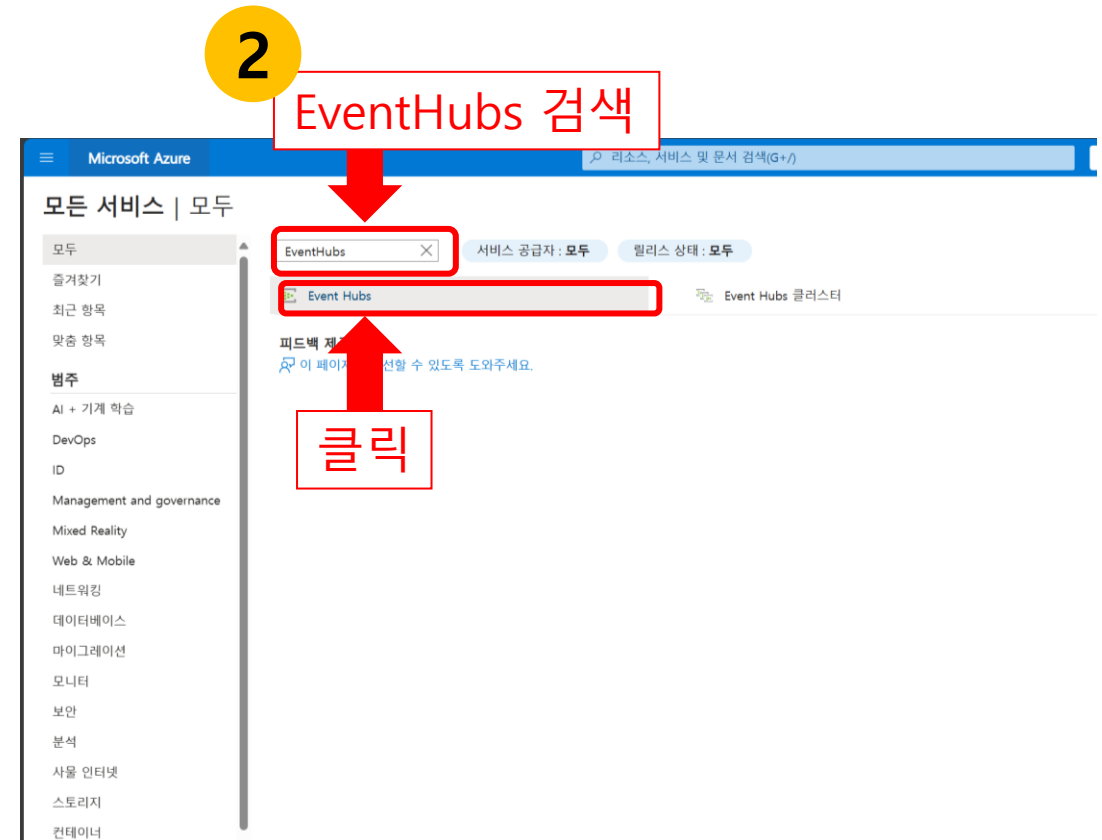
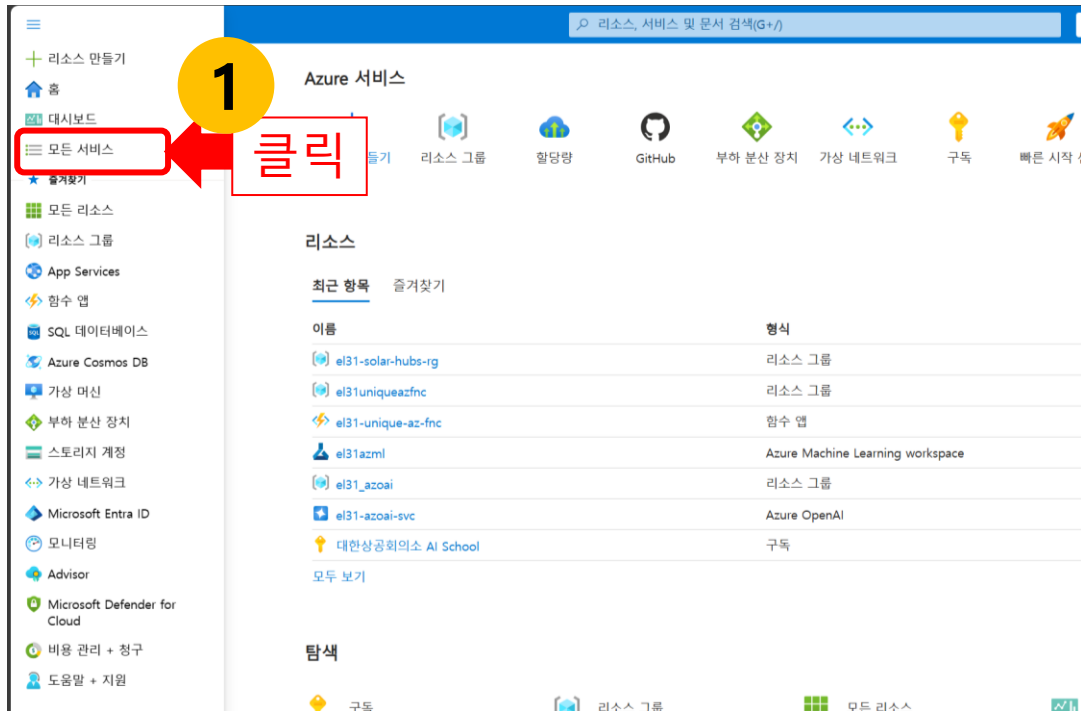
클릭

플랫 다운로드

Event Hubs 네임스페이스 생성(1)

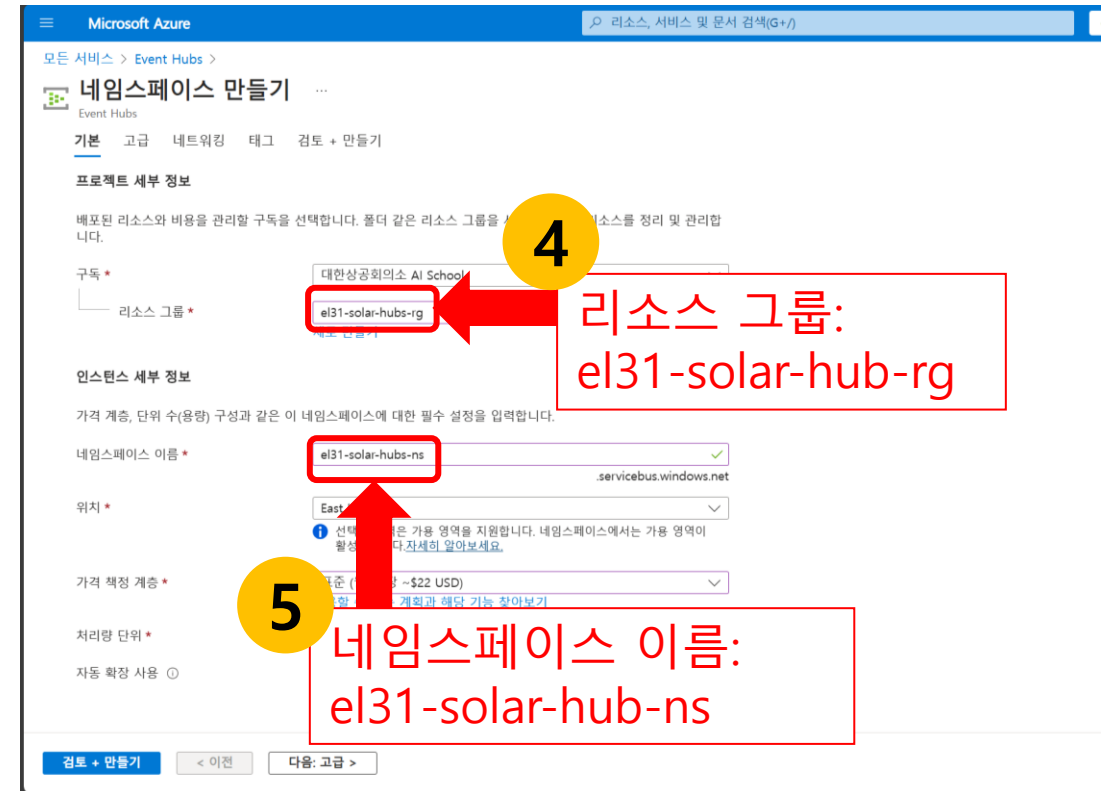
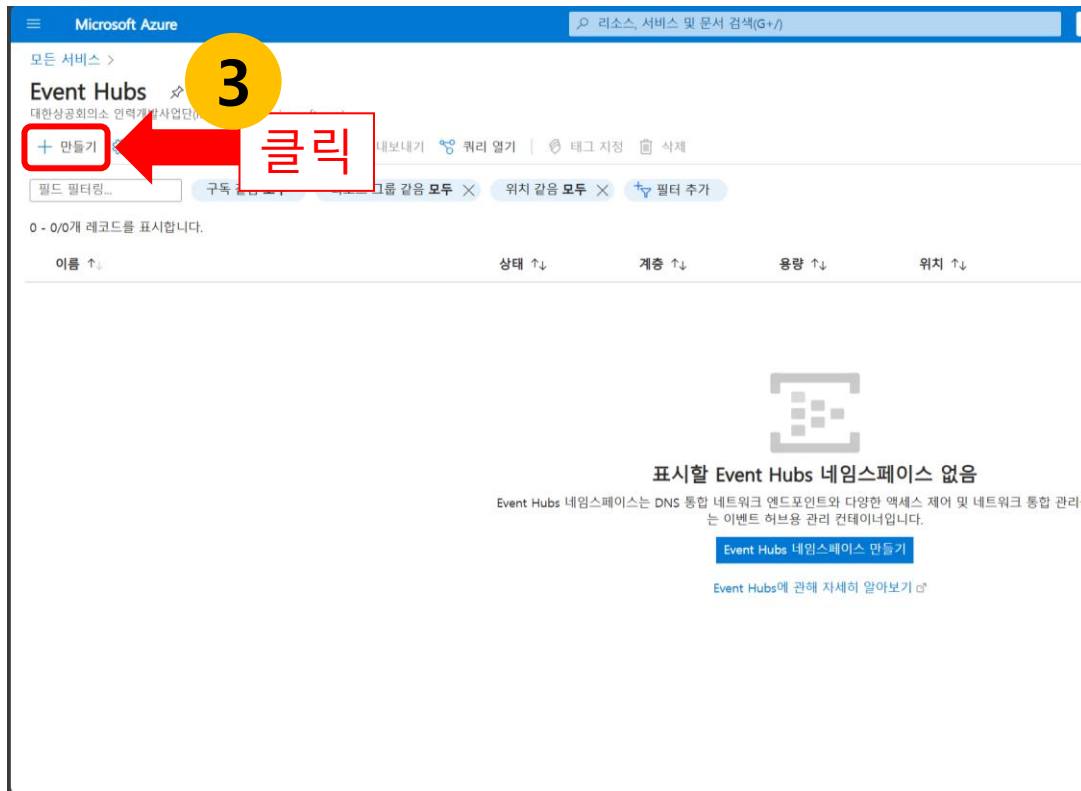
Event Hubs 네임스페이스는 Event Hub를 포함하는 상위 개념입니다, 하나의 네임스페이스 안에 여러 개의 Event Hub를 만들 수 있습니다.

1. Azure Portal의 '왼쪽' 메뉴에서 '모든 서비스' 메뉴 선택합니다.
2. EventHubs를 검색하고 선택합니다.



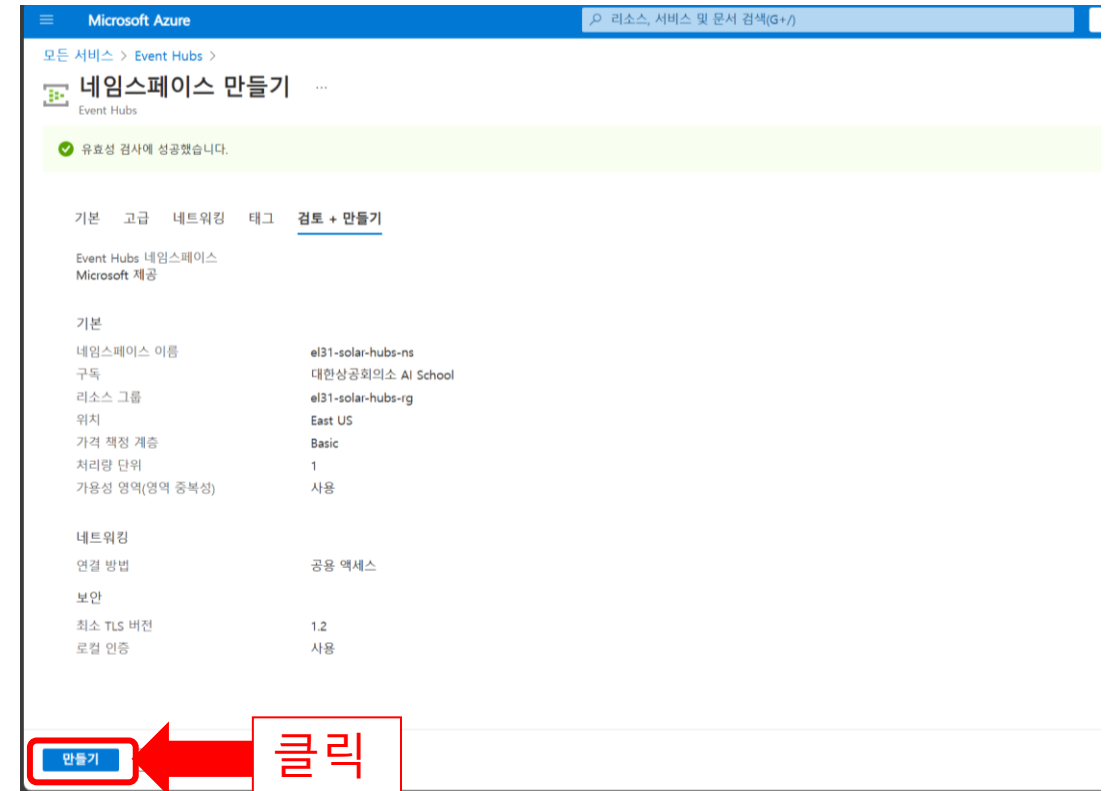
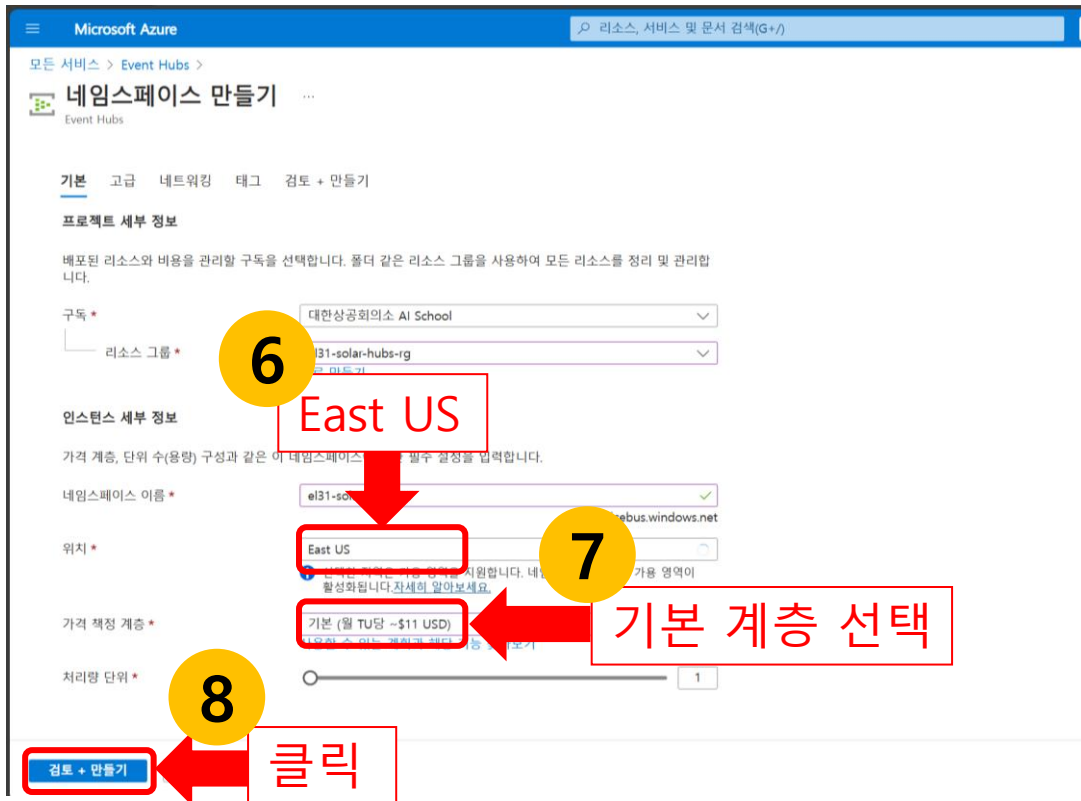
Event Hubs 네임스페이스 생성(2)

3. 'Event Hubs' 페이지에서 '+ 만들기' 버튼을 클릭합니다.
4. 이벤트 허브를 만들 구독과 리소스 그룹을 선택 합니다.(앞서 만든 리소스 그룹)
5. 네임스페이스 이름은 고유한 이름을 입력합니다. (예: el31-solar-hub-ns)



Event Hubs 네임스페이스 생성(3)

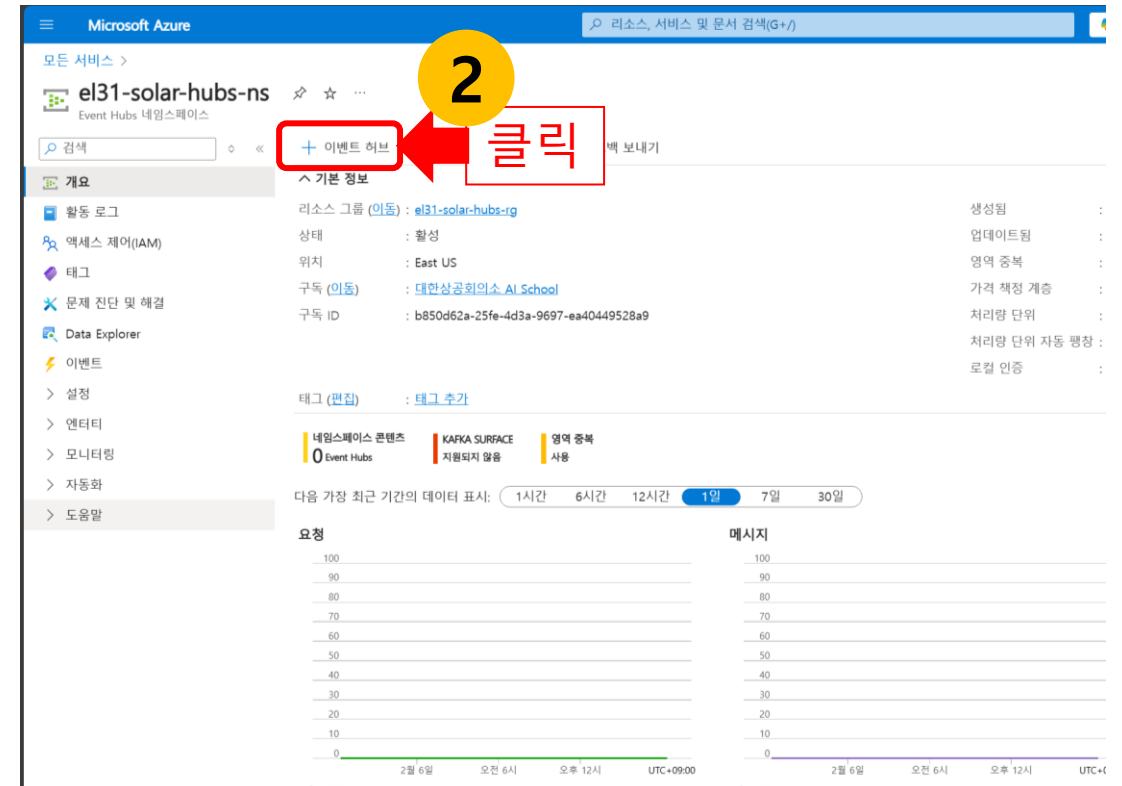
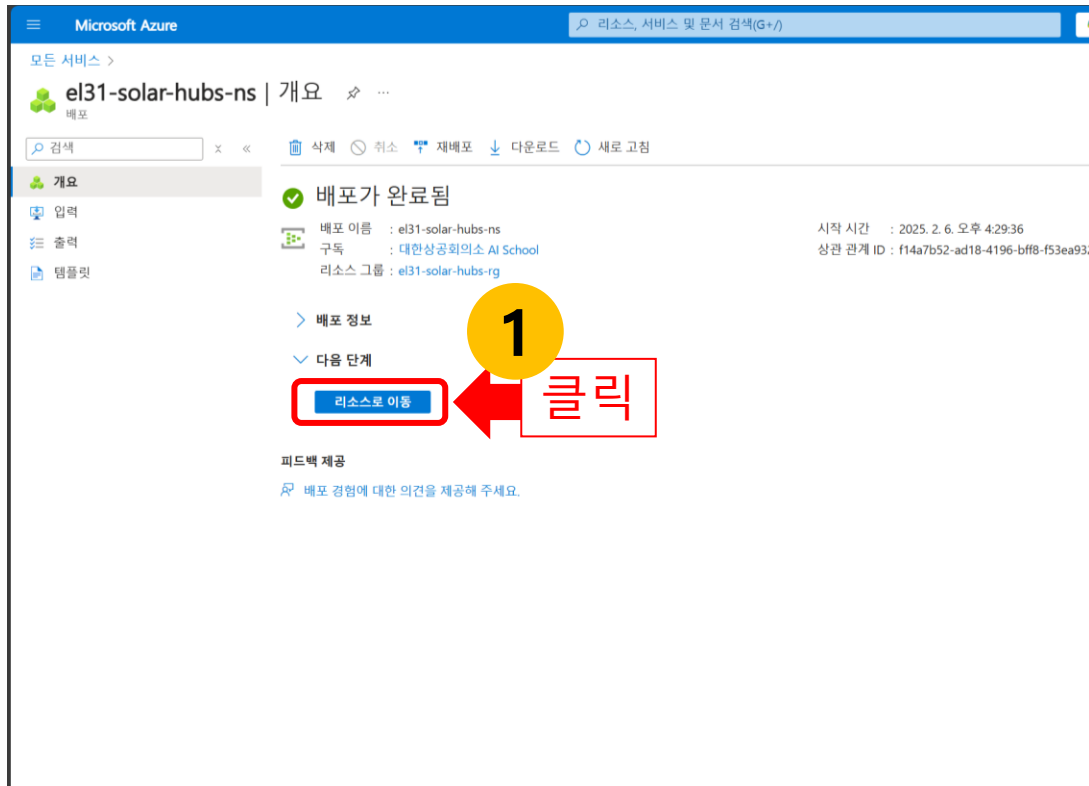
6. 위치는 East US 선택합니다.
7. 가격 책정 계층은 기본 계층을 선택합니다.
8. '검토 + 만들기' 버튼을 클릭 후 '만들기' 버튼을 클릭합니다.



Event Hub 생성(1)

드디어 Event Hub를 만들 시간입니다. Event Hub는 데이터가 실제로 흐르는 통로입니다.

1. '네임스페이스 배포 완료' 페이지에서 '리소스로 이동 버튼'을 클릭합니다.
2. '네임스페이스 개요' 페이지에서 '+ 이벤트 허브' 버튼을 클릭합니다.



Event Hub 생성(2)

- 이벤트 허브 이름은 고유한 이름을 입력합니다. (예: el31_solar_hub)
- 파티션 수는 1개를 선택합니다.
- 메시지 보존 정책은 삭제 그리고 메시지 보존 시간(1시간)을 선택합니다.
- '검토 + 만들기' 버튼을 클릭 후 '만들기' 버튼을 클릭합니다.

Microsoft Azure

모든 서비스 > el31-solar-hubs-ns > 이벤트 허브 만들기

이벤트 허브 만들기

기본 탭 검토 + 만들기

이벤트 허브 세부 정보

파티션 수 및 메시지 보존을 포함하여 이 이벤트 허브에 필요한 설정을 입력합니다.

이름 *

파티션 수

보존

이 이벤트 허브에 대한 보존 설정을 구성합니다. 자세한 정보

정리 정책

보존 시간(시간) *

검토 + 만들기

클릭

3

4

1개

5

- 메시지 보존 정책: 삭제
- 메시지 보존 시간: 1시간

Microsoft Azure

모든 서비스 > el31-solar-hubs-ns > 이벤트 허브 만들기

이벤트 허브 만들기

Event Hubs

유효성 검사에 성공했습니다.

기본 탭 검토 + 만들기

Event Hubs 인스턴스

Microsoft 제공

기본	
이름	el31_solar_hub
파티션 수	1
보존	
정리 정책	Delete
보존 시간(시간)	1
캡처	
캡처 상태	지원되지 않음

만들기

클릭

3

4

1개

5

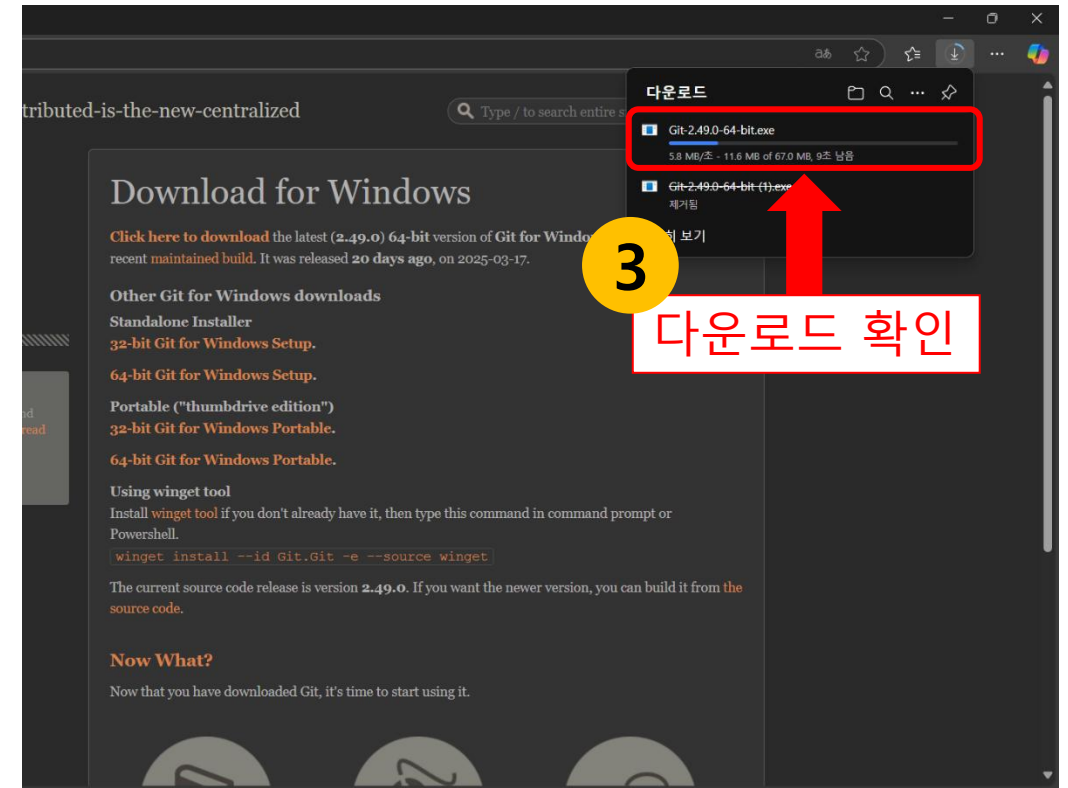
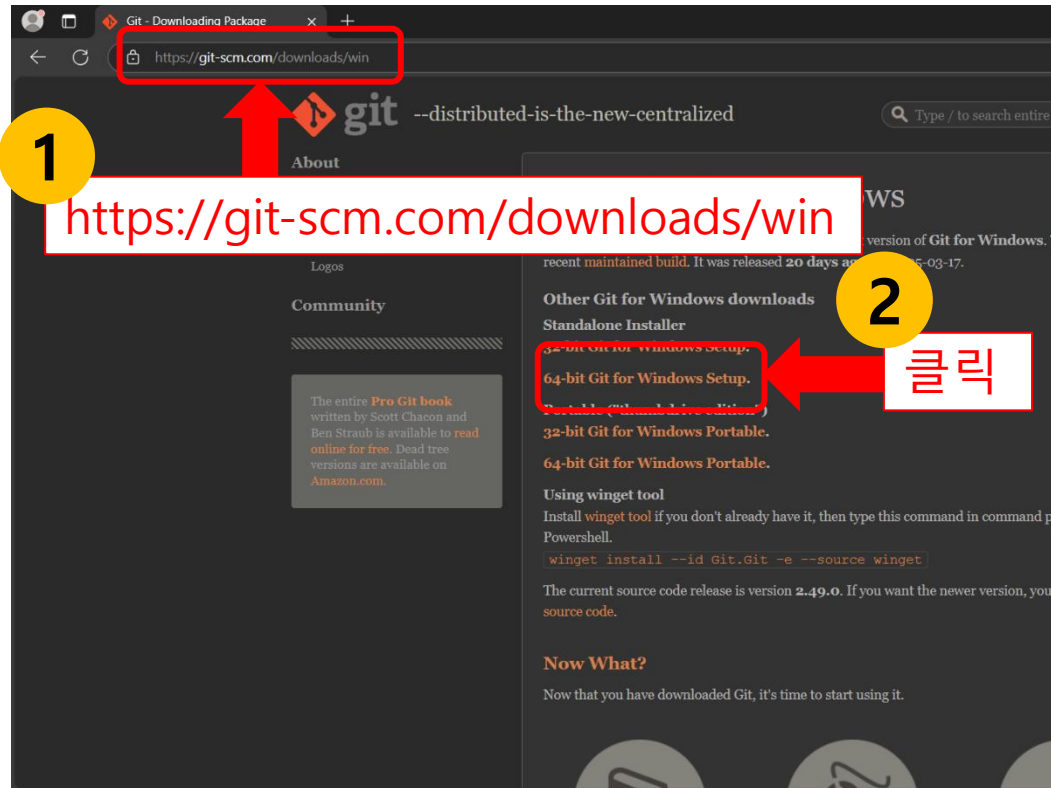
- 메시지 보존 정책: 삭제
- 메시지 보존 시간: 1시간

Azure Functions를 사용하여 Event Hubs 전달

Git 설치 파일 다운로드하기

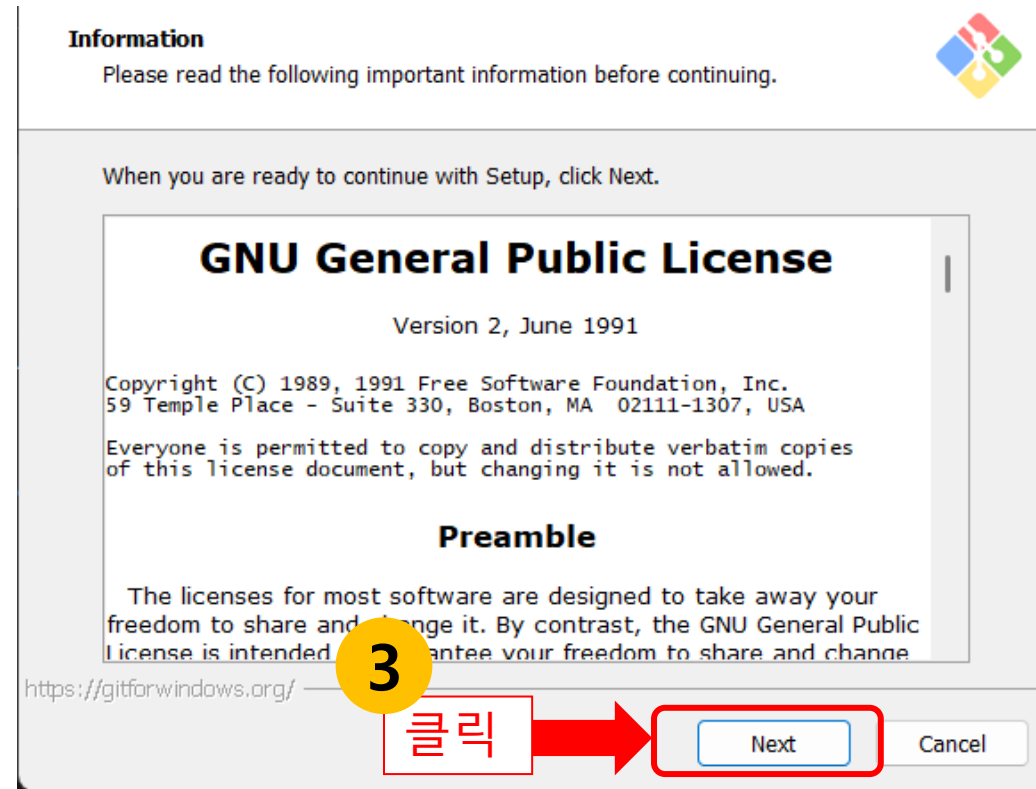
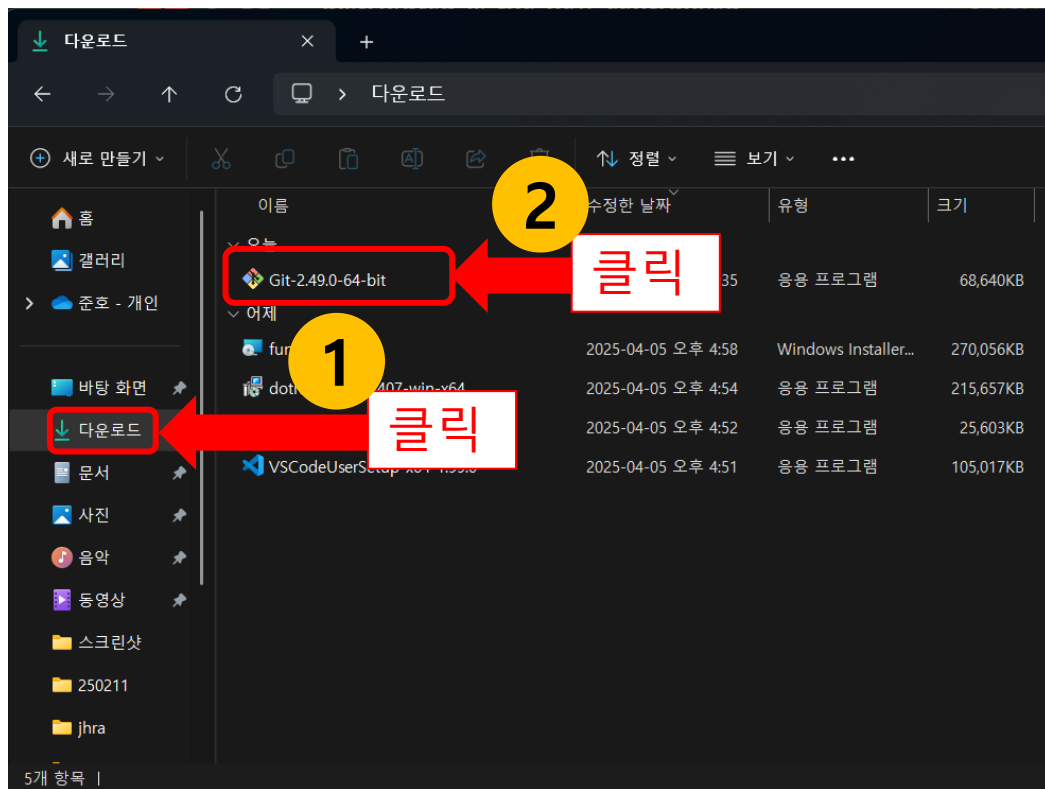
Git 설치 파일을 다운로드 하는 방법에 대해 알아보겠습니다.

1. Git 다운로드 페이지 접속 (<https://git-scm.com/downloads/win>)
2. '64-bit Git for Windows Setup' 버튼을 클릭합니다.
3. Git 설치 파일이 다운로드 되는 것을 확인합니다. (예: Git-2.49.0-64-bit.exe)



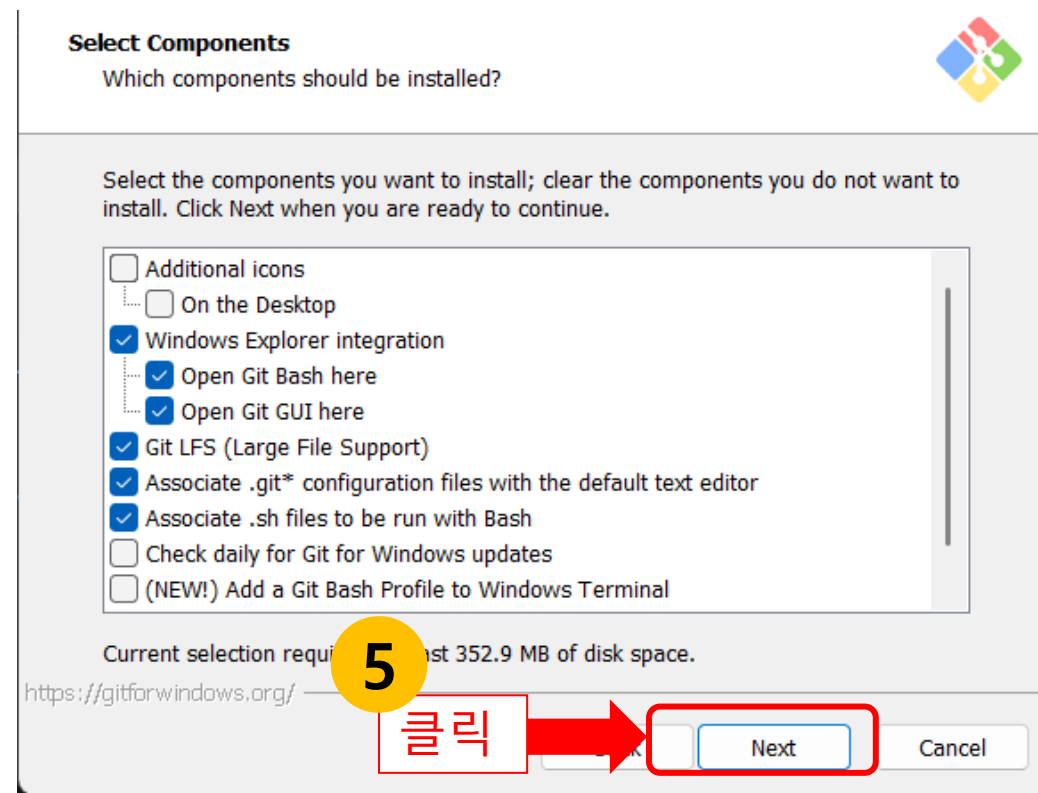
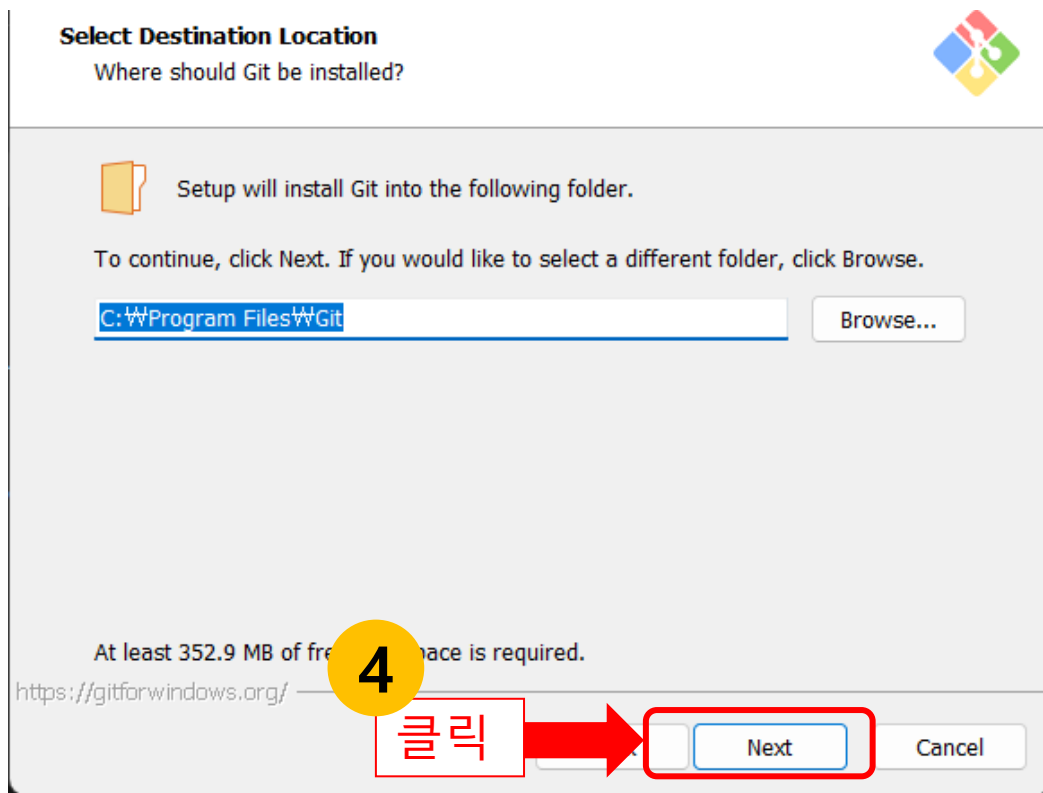
Git 설치하기(1)

1. 파일 탐색기에서 '다운로드' 폴더를 클릭합니다.
2. 설치된 Git exe 파일을 실행합니다. (예: Git-2.49.0-64-bit.exe)
3. 'Git 설치' 팝업창에서 'Next' 버튼을 클릭합니다.



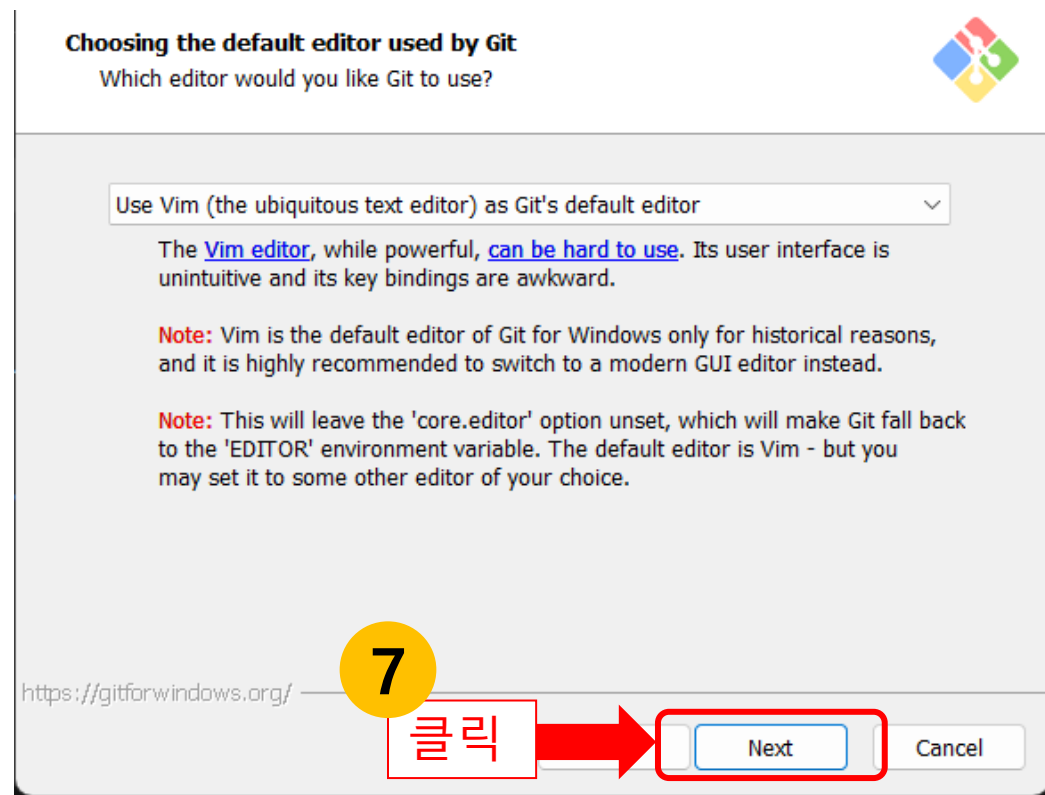
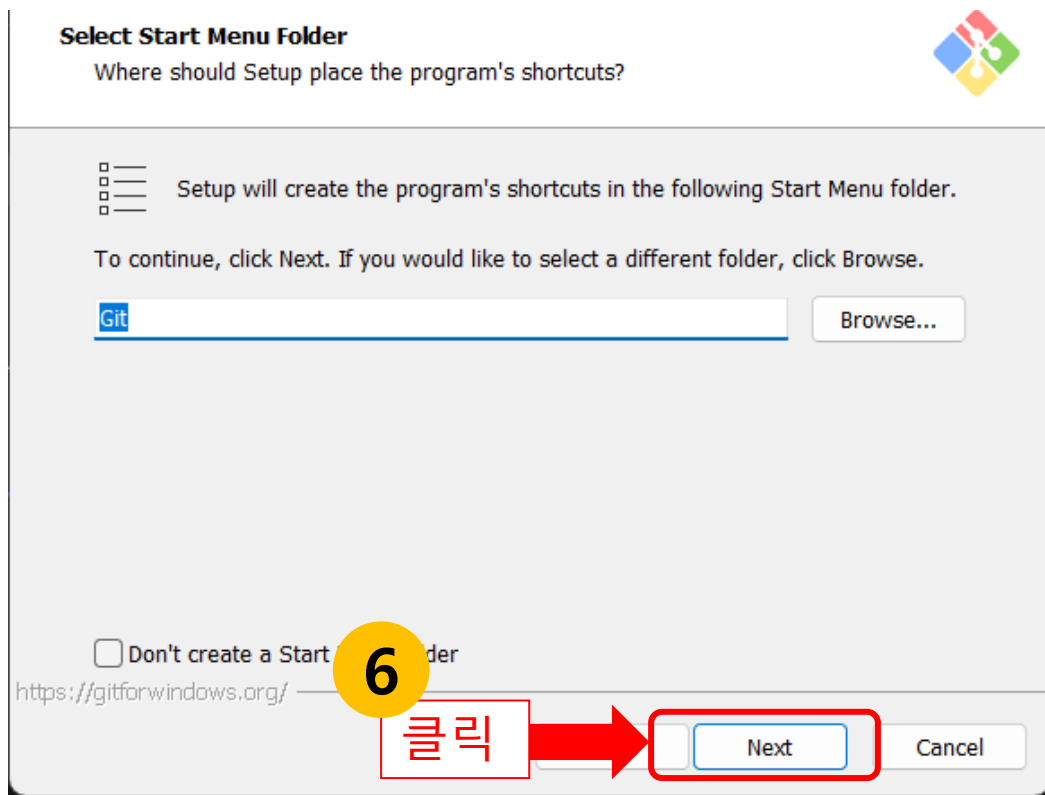
Git 설치하기(2)

4. 'Select Destination Location' 화면에서 'Next' 버튼을 클릭합니다.
5. 'Select Components' 화면에서 'Next' 버튼을 클릭합니다.



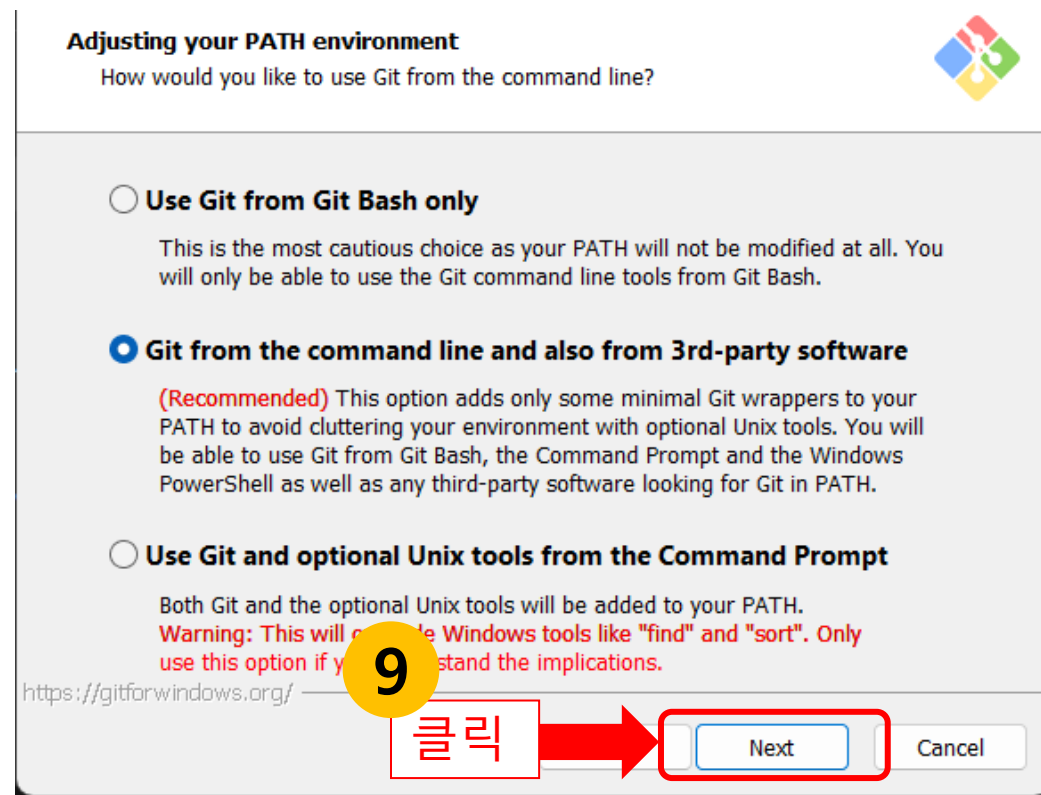
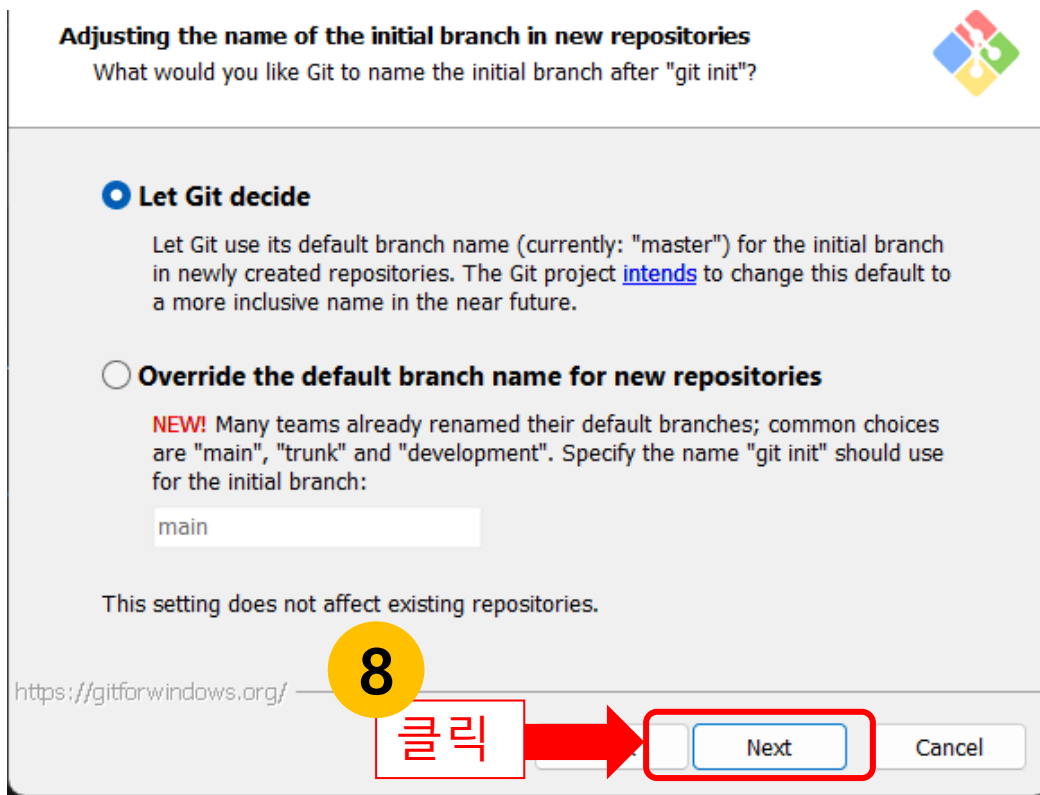
Git 설치하기(3)

6. 'Select Start Menu Folder' 화면에서 'Next' 버튼을 클릭합니다.
7. 'Choosing the default editor used by Git' 화면에서 'Next' 버튼을 클릭합니다.



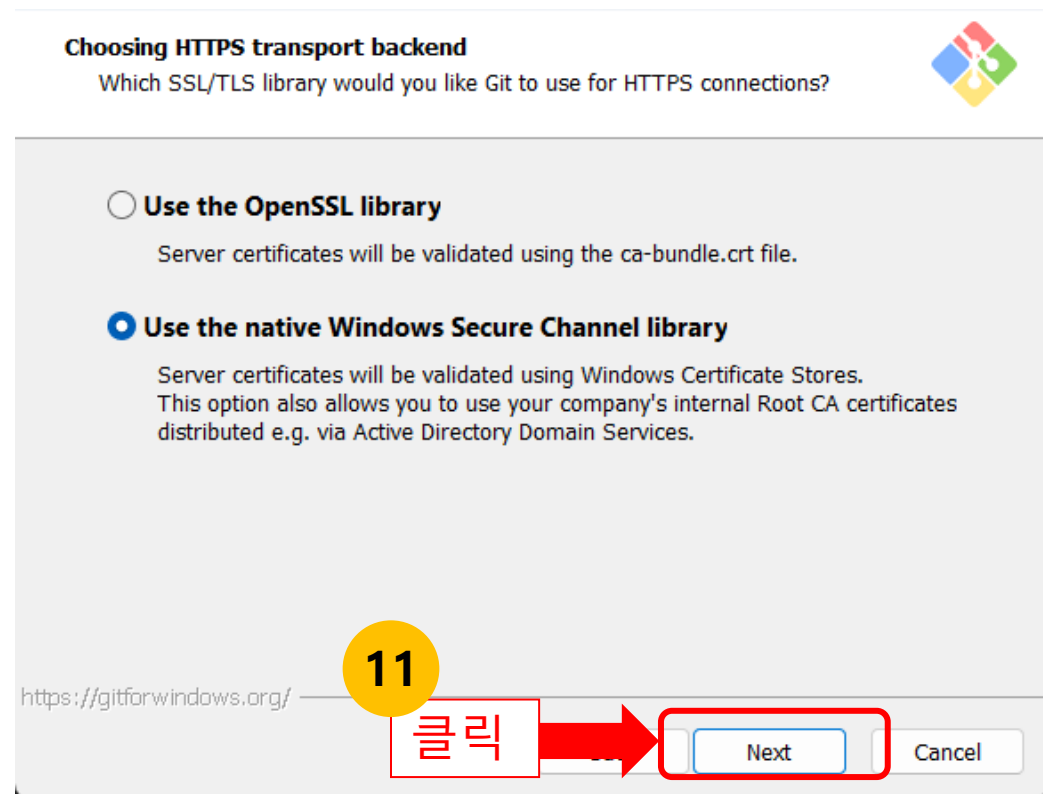
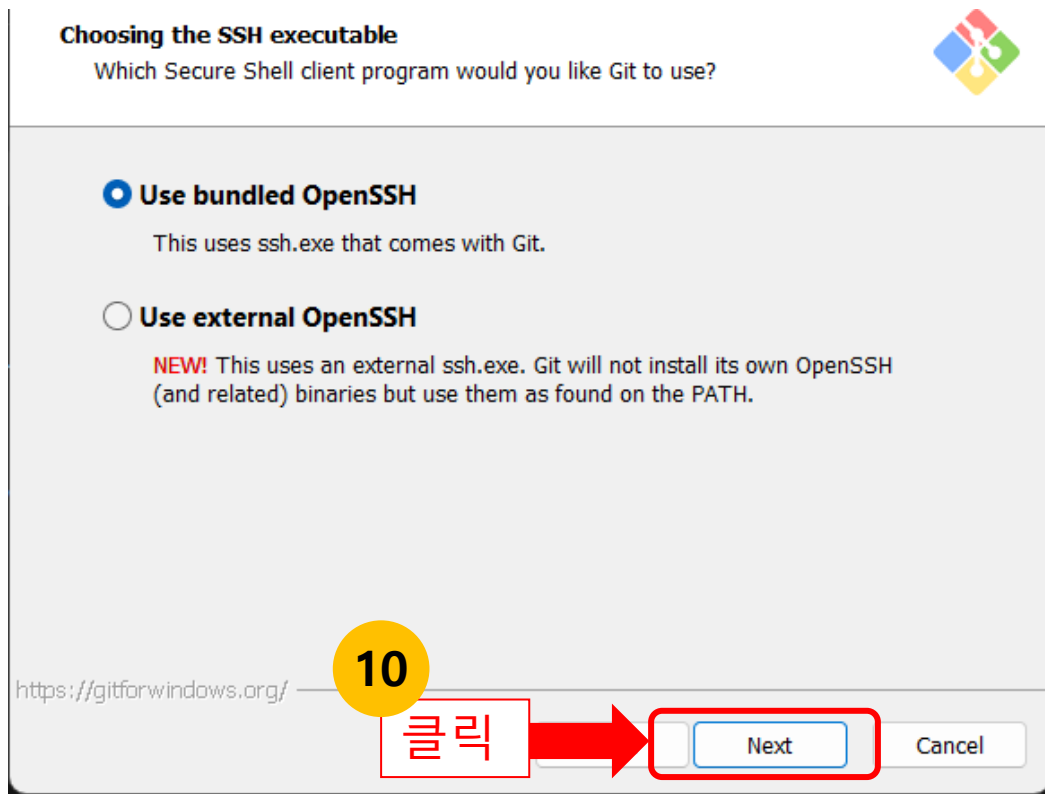
Git 설치하기(4)

8. 'Adjusting the name of the initial branch in new repositories' 화면에서 'Next' 버튼을 클릭합니다.
9. 'Adjusting your PATH environment' 화면에서 'Next' 버튼을 클릭합니다.



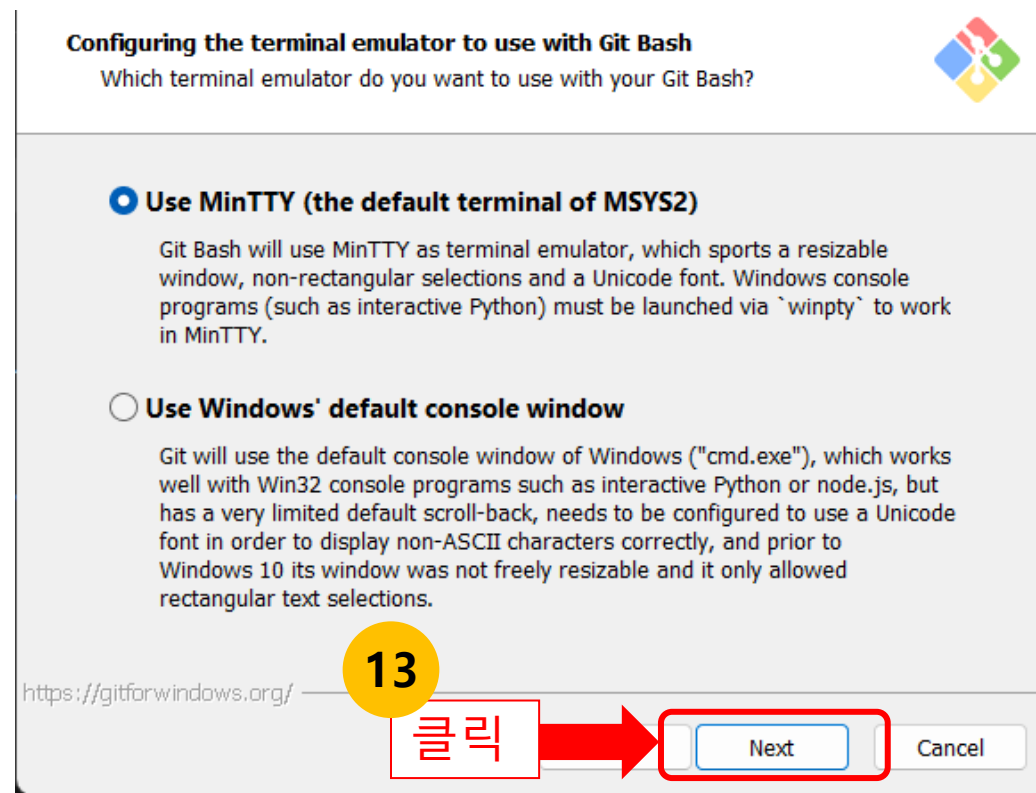
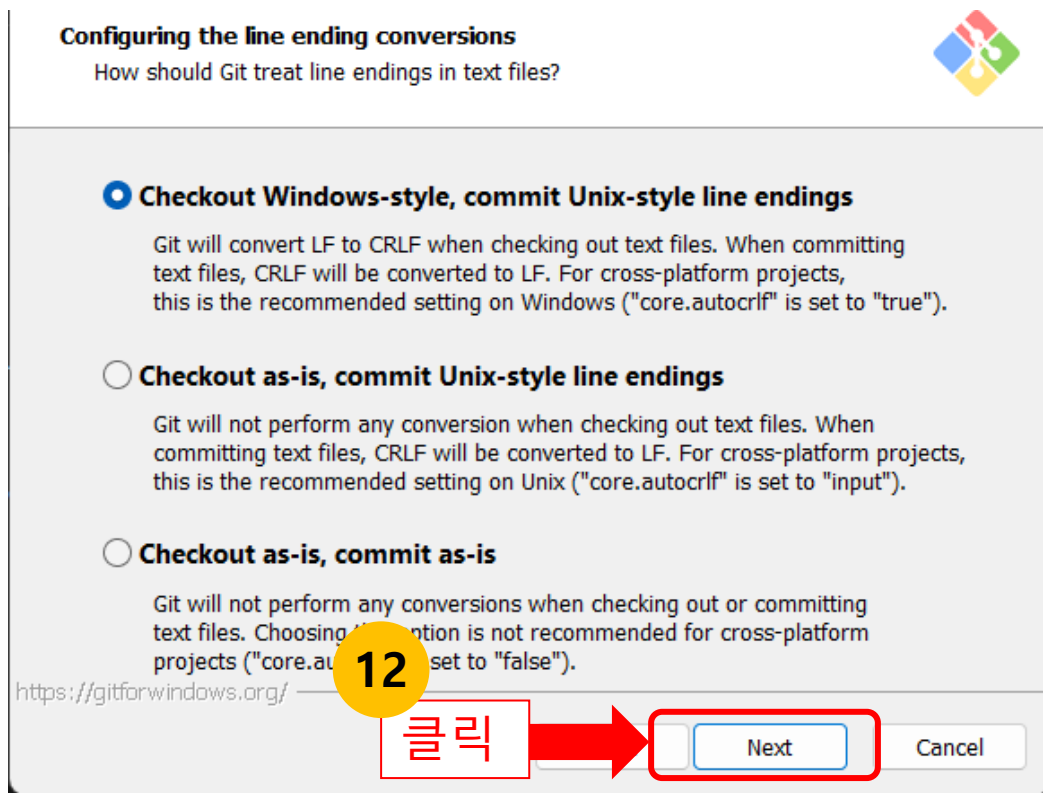
Git 설치하기(5)

10. 'Choosing the SSH executable' 화면에서 'Next' 버튼을 클릭합니다.
11. 'Choosing HTTPS transport backend' 화면에서 'Next' 버튼을 클릭합니다.



Git 설치하기(6)

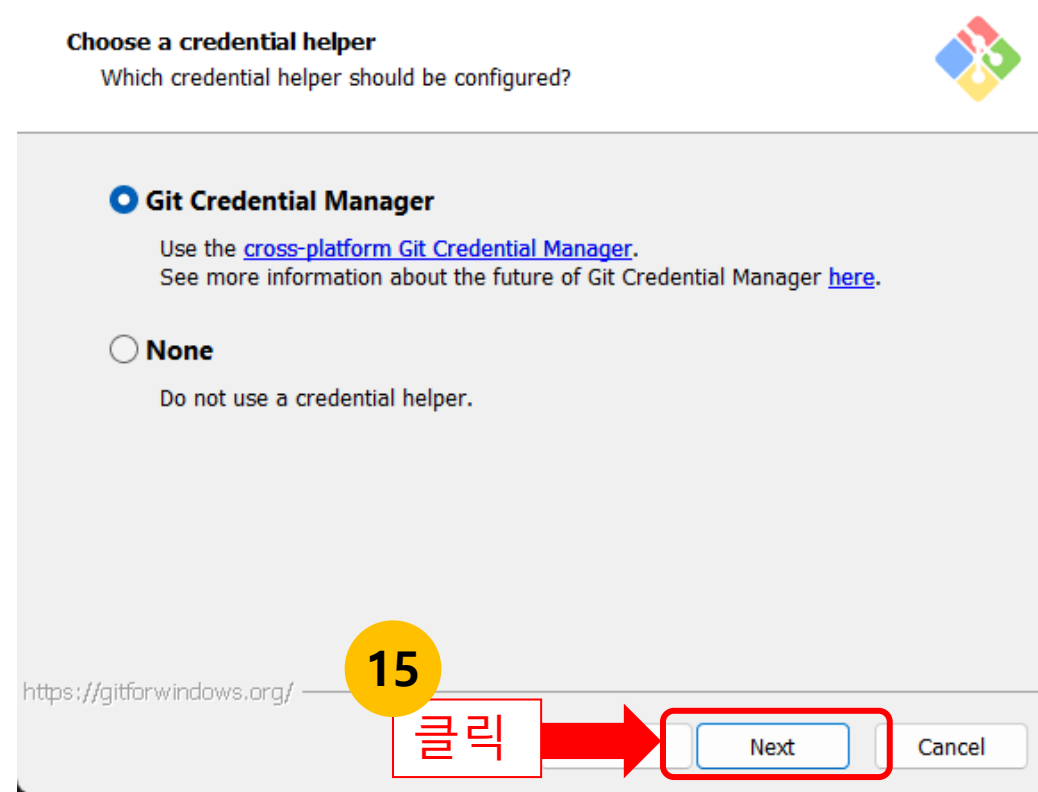
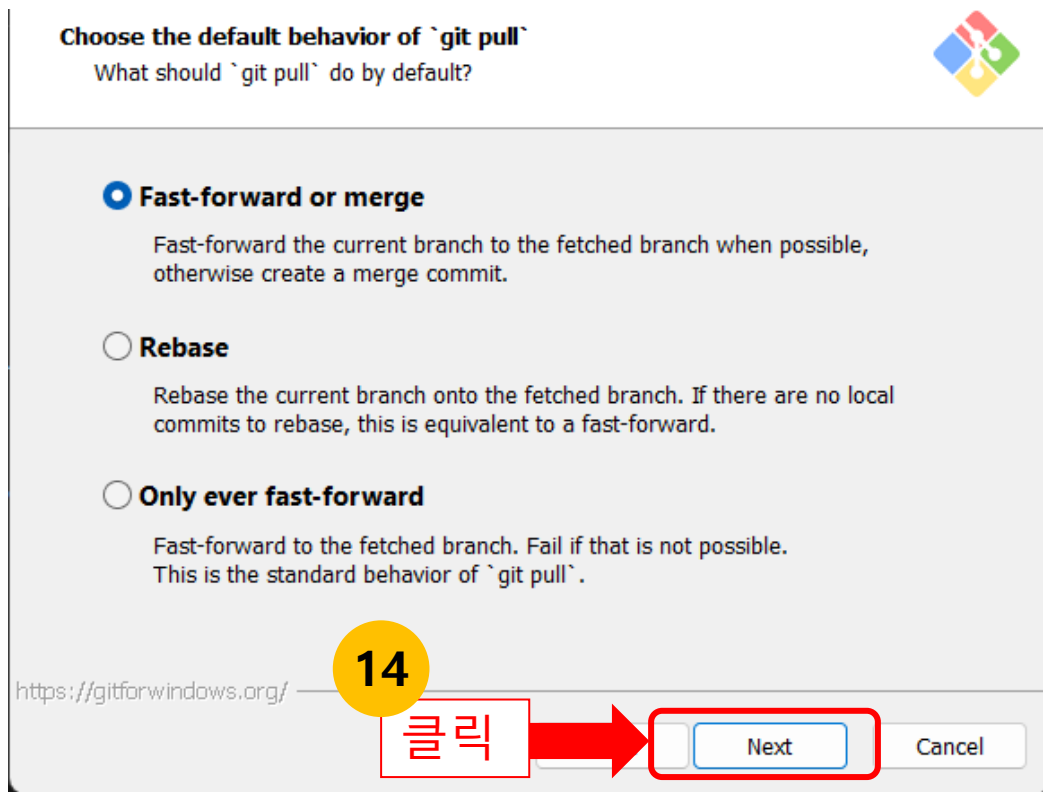
12. 'Configuring the line ending conversions' 화면에서 'Next' 버튼을 클릭합니다.
13. 'Configuring the terminal emulator to use with Git Bash' 화면에서 'Next' 버튼을 클릭합니다.



Git 설치하기(7)

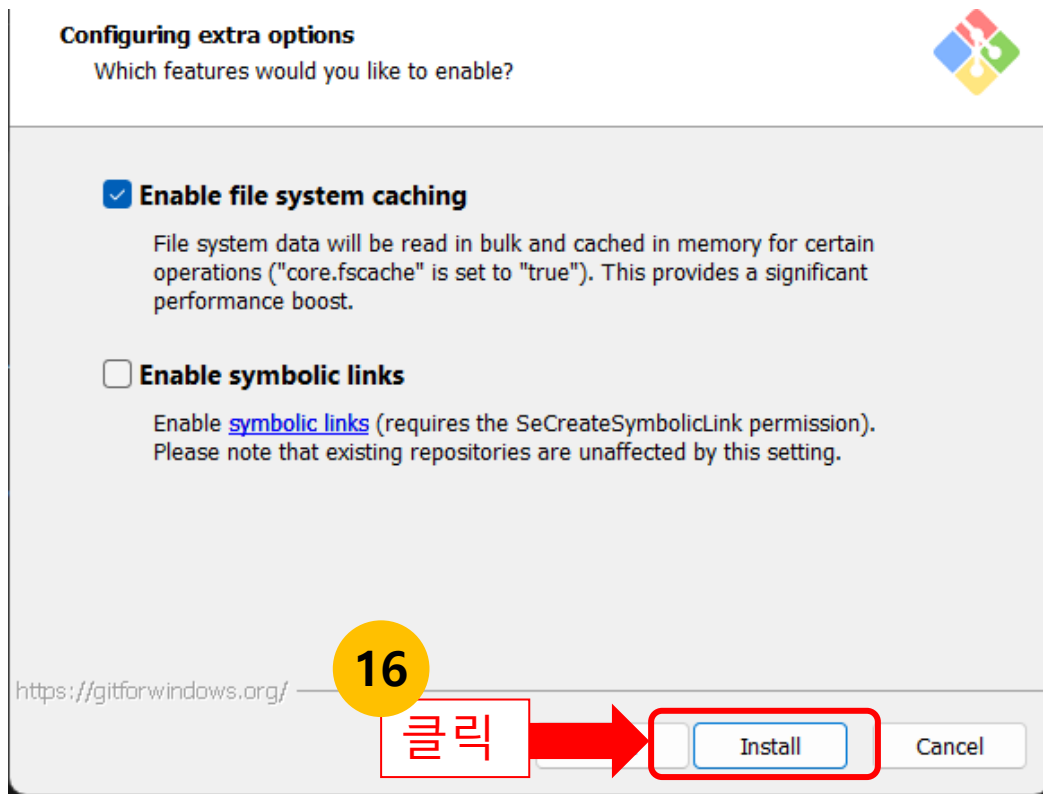
14. 'Choose the default behavior of git pull' 화면에서 'Next' 버튼을 클릭합니다.

15. 'Choose a credential helper' 화면에서 'Next' 버튼을 클릭합니다.



Git 설치하기(8)

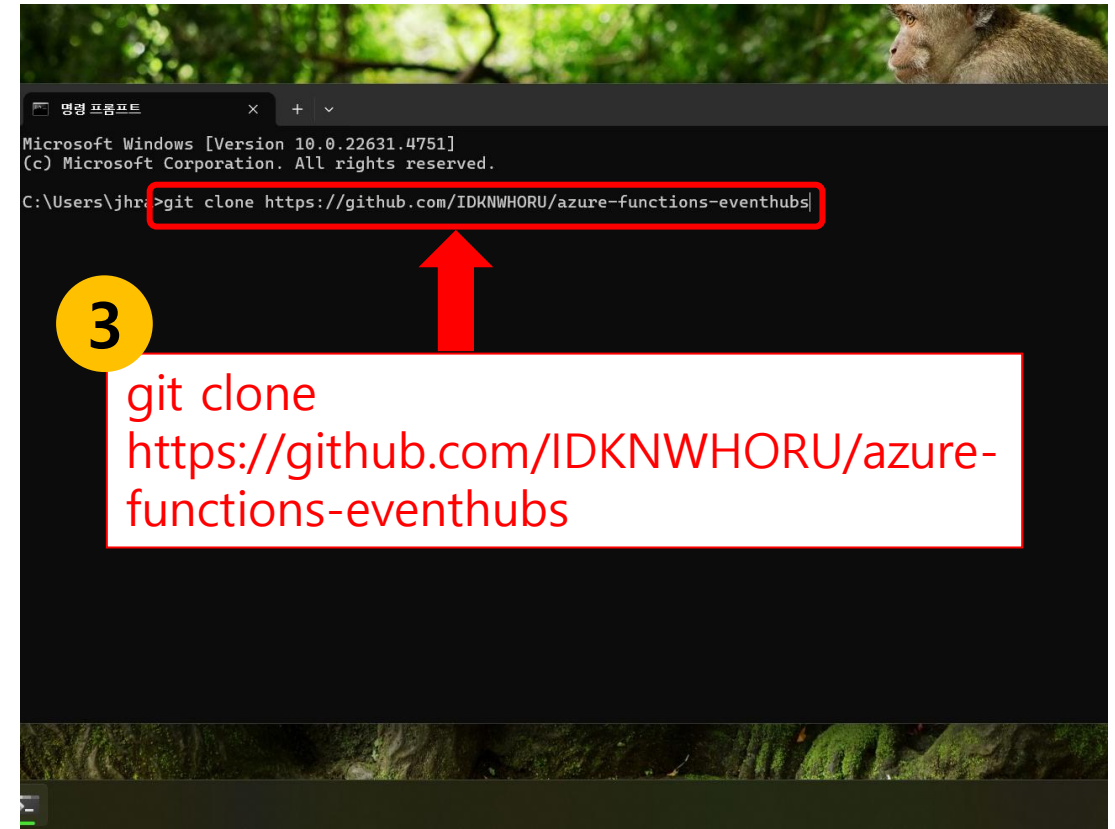
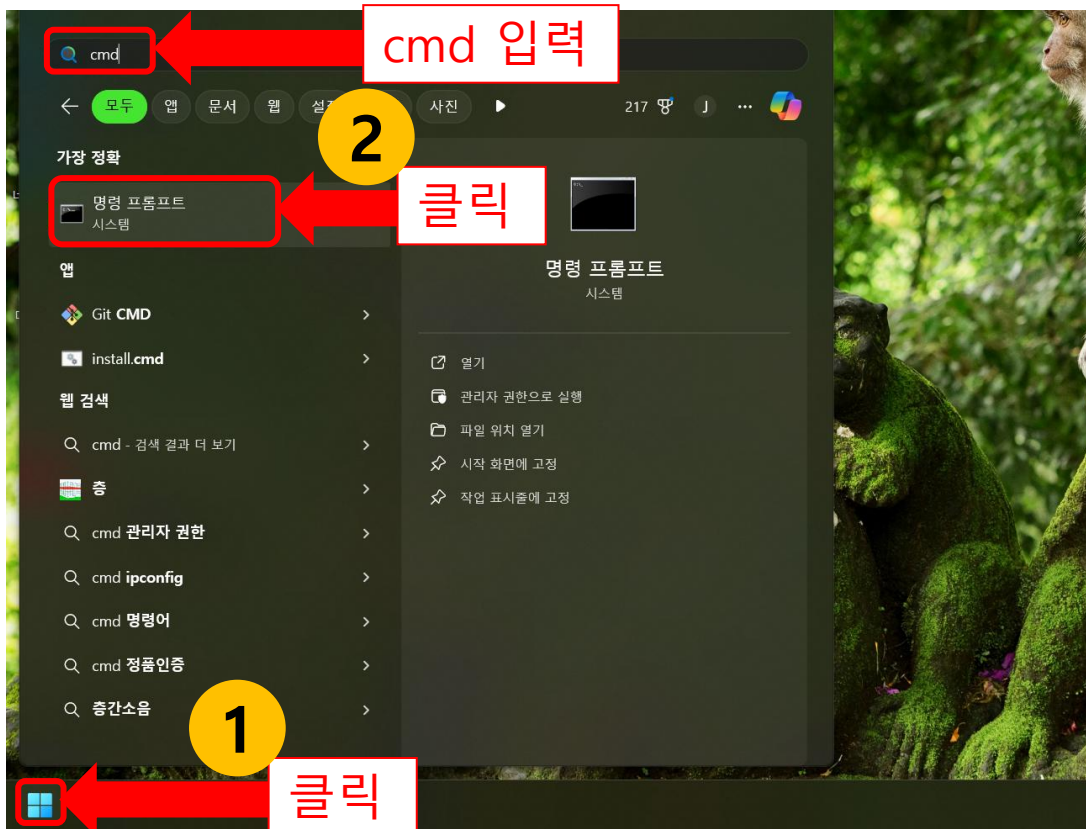
16. 'Configuring extra options' 화면에서 'Install' 버튼을 클릭합니다.
17. 'Completing the Git Setup Wizard' 화면에서 'View Release Notes' 체크 박스를 클릭해 해제 상태로 변경합니다.
18. 'Finish' 버튼을 클릭합니다.



Git을 이용해 Azure Function EventHubs 실습 프로젝트 클론하기(1)

실습 프로젝트를 Git을 통해 클론하는 방법에 대해 자세히 알아보겠습니다.

1. 'windows' 키를 입력 후 cmd를 입력합니다.
2. '명령 프롬프트' 프로그램을 실행합니다.
3. `git clone https://github.com/IDKNWHORU/azure-functions-eventhubs` 를 입력하고 실행합니다.



Git을 이용해 Azure Function EventHubs 실습 프로젝트 클론하기(2)

4. `cd .\azure-functions-eventhubs\` 를 입력하고 실행합니다.

5. `code .` 를 입력하고 실행합니다.

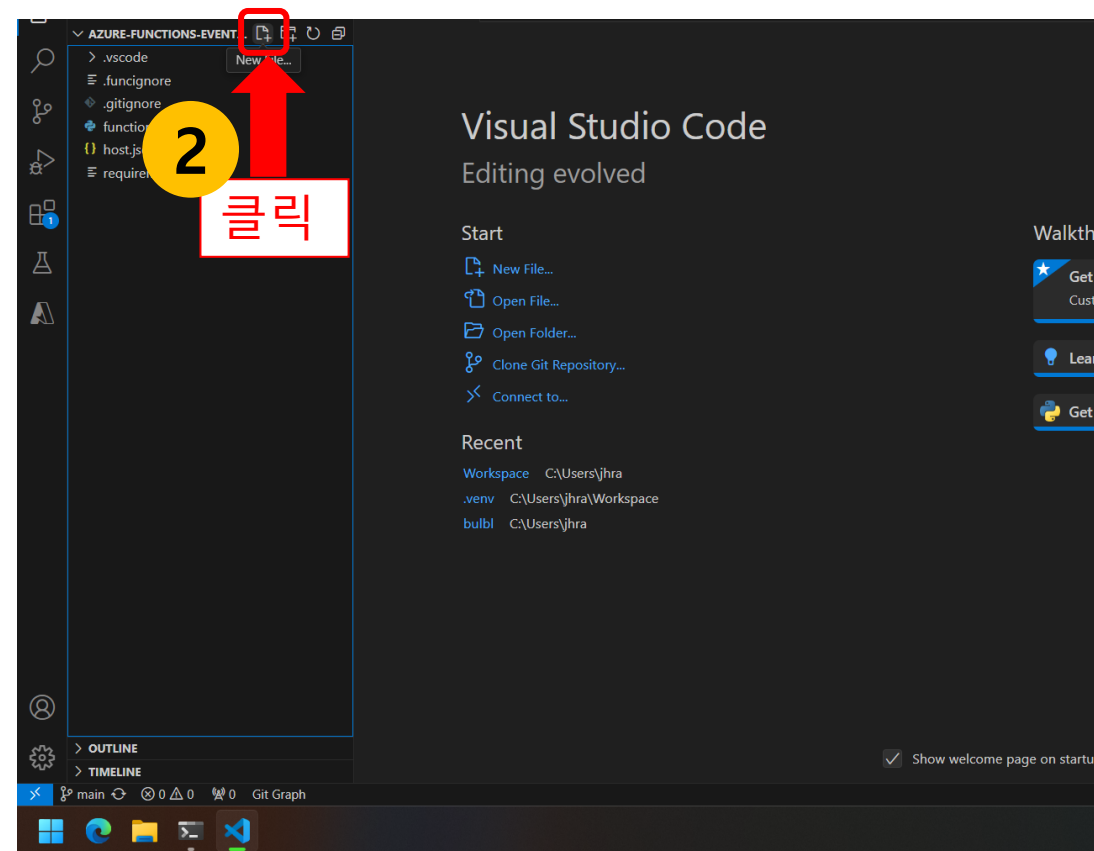
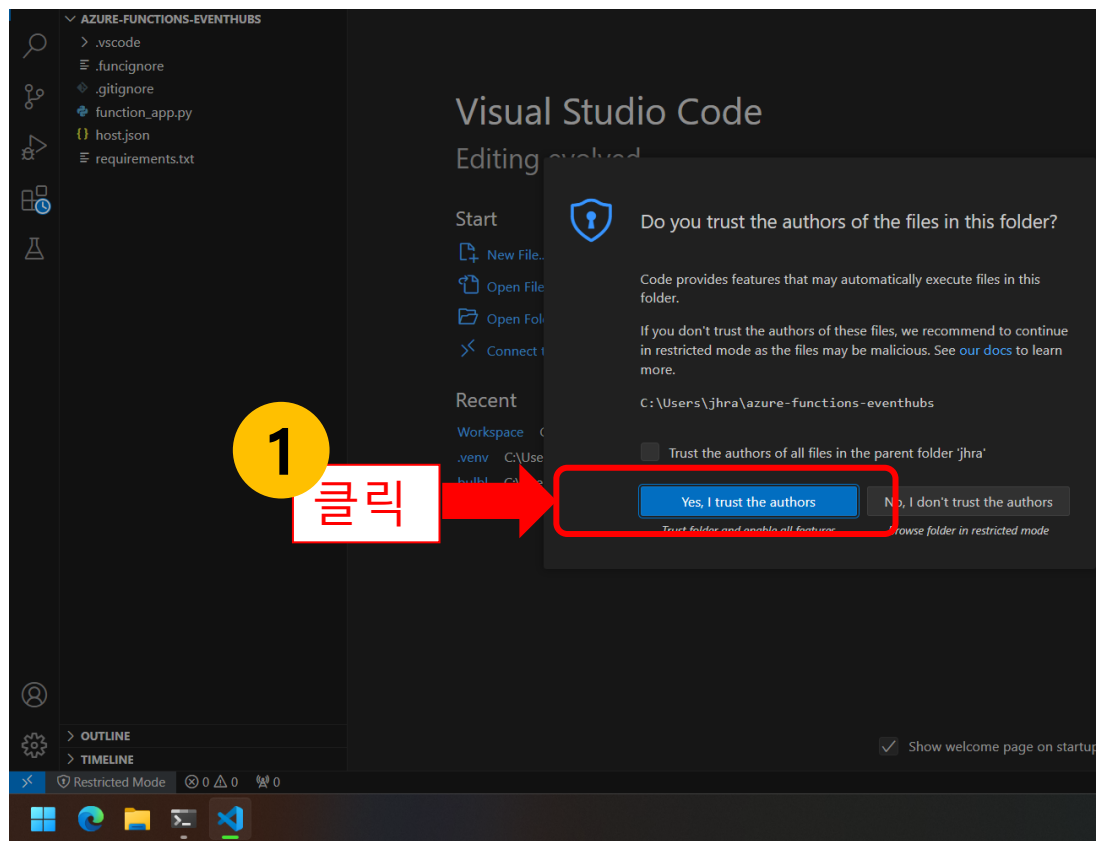
A screenshot of a Windows PowerShell terminal window. The terminal shows the command `cd .\azure-functions-eventhubs\` being entered. A red box highlights the command, and a red arrow points from a yellow circle with the number '4' to the box. Below the terminal, a white box with a red border contains the text `cd .\azure-functions-eventhubs\`.

A screenshot of a Windows PowerShell terminal window. The terminal shows the command `code .` being entered. A red box highlights the command, and a red arrow points from a yellow circle with the number '5' to the box. Below the terminal, a white box with a red border contains the text `code .`.

VS Code에서 Azure Functions 프로젝트 기본 설정하기(1)

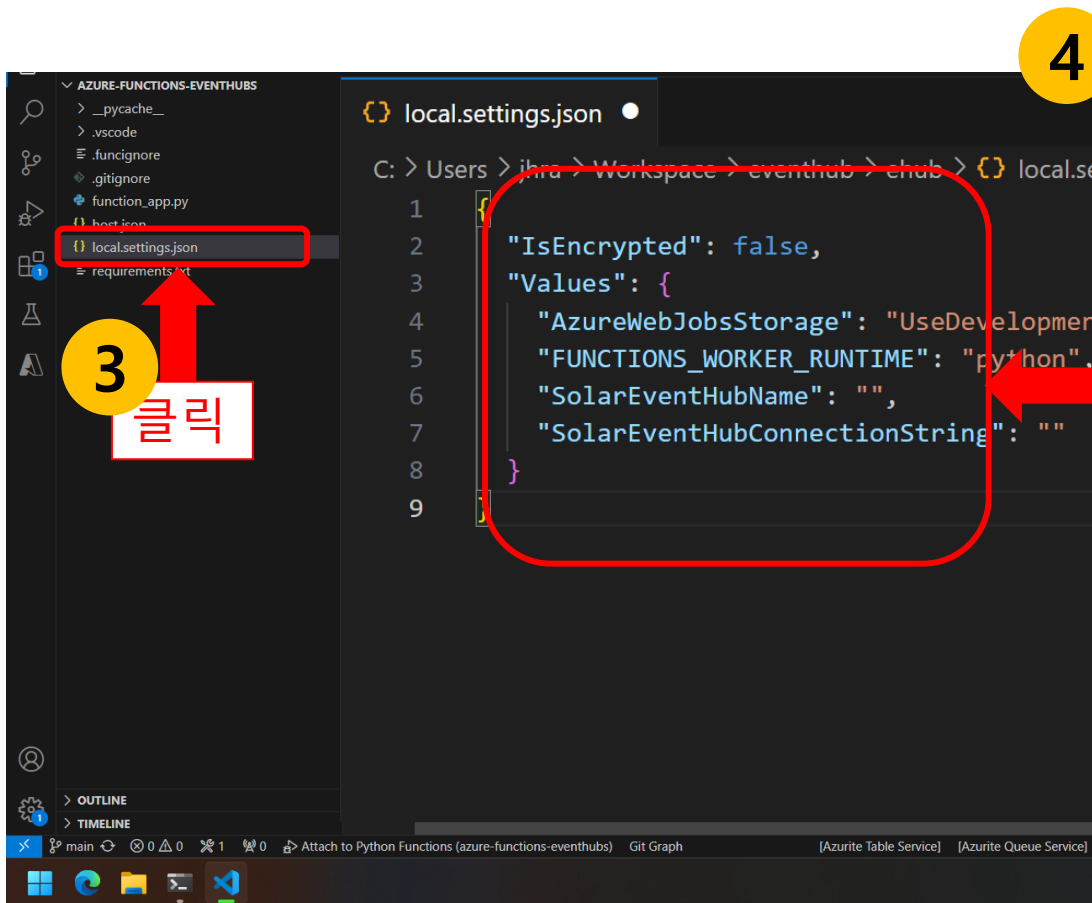
VS Code를 활용하여 서버리스 애플리케이션을 개발하는 과정에서, 프로젝트의 기본 설정은 매우 중요합니다.

1. 'Yes, I trust the authors' 버튼을 클릭합니다.
2. '새 폴더 추가' 버튼을 클릭합니다.



VS Code에서 Azure Functions 프로젝트 기본 설정하기(2)

3. 새 폴더의 이름을 **local.settings.json** 으로 입력합니다.
4. **local.settings.json** 파일에 필요한 설정 정보를 입력합니다.



The screenshot shows the VS Code interface. In the Explorer on the left, the file `local.settings.json` is highlighted with a red box. A yellow circle with the number '3' and a red arrow points to it, with the Korean text '클릭' (click) below. In the Editor, the `local.settings.json` file is open, showing the following JSON content:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "python",
    "SolarEventHubName": "",
    "SolarEventHubConnectionString": ""
  }
}
```

A red box highlights the JSON content, and a yellow circle with the number '4' and a red arrow points to it.

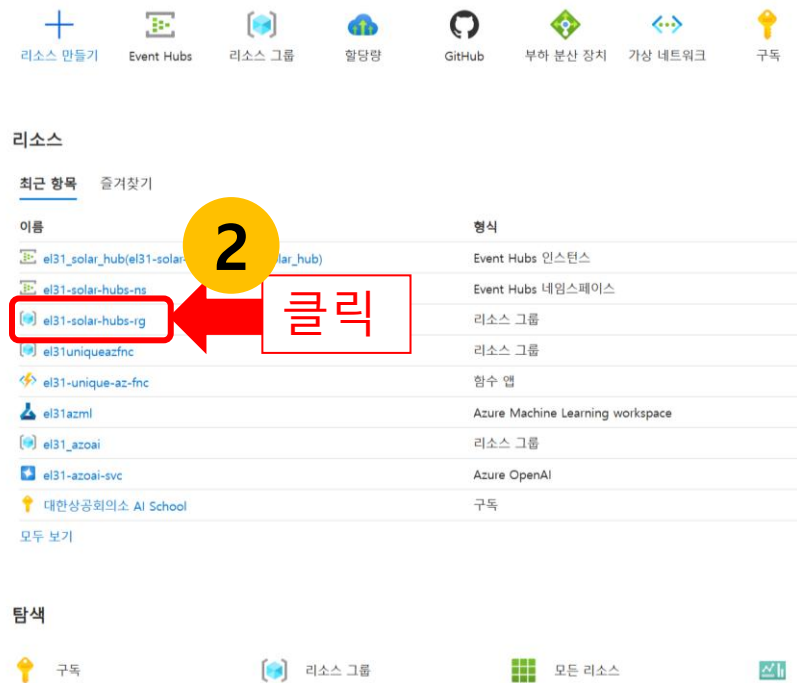
```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "python",
    "SolarEventHubName": "",
    "SolarEventHubConnectionString": ""
  }
}
```


Event Hubs 연결 문자열 설정하기(1)

연결 문자열은 애플리케이션이 Event Hub에 접근할 수 있도록 해주는 인증 정보를 포함하고 있습니다.

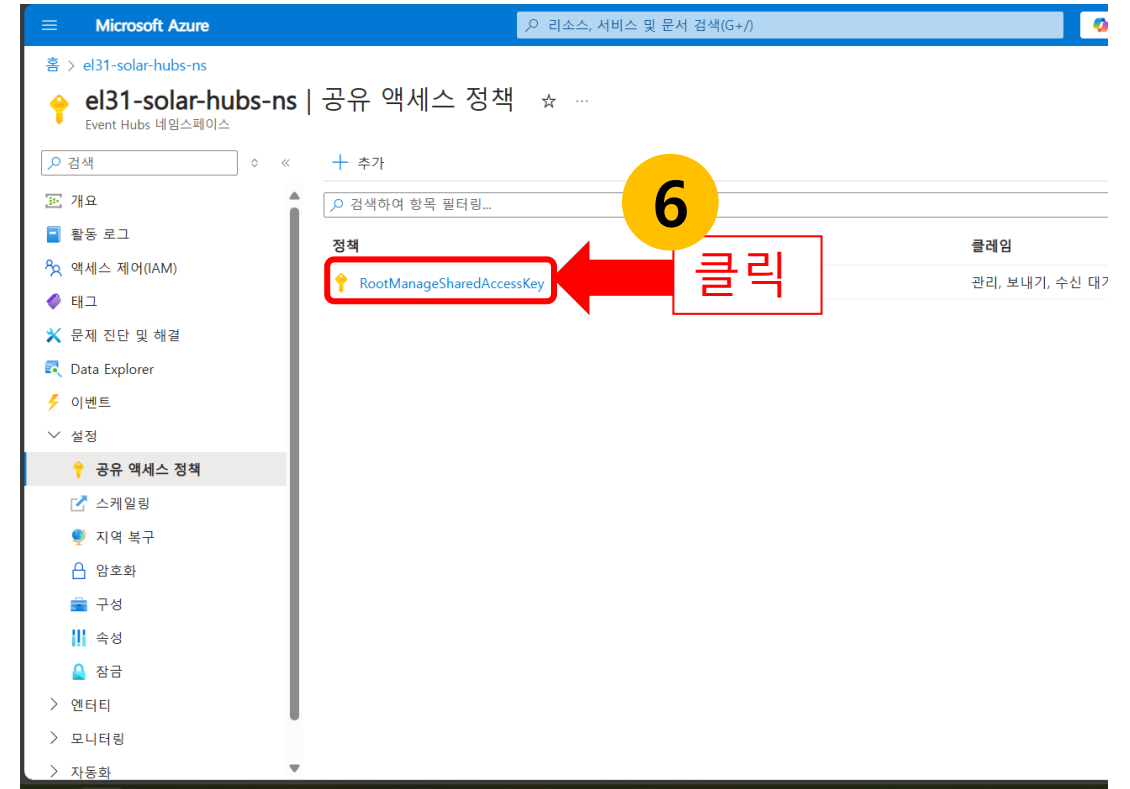
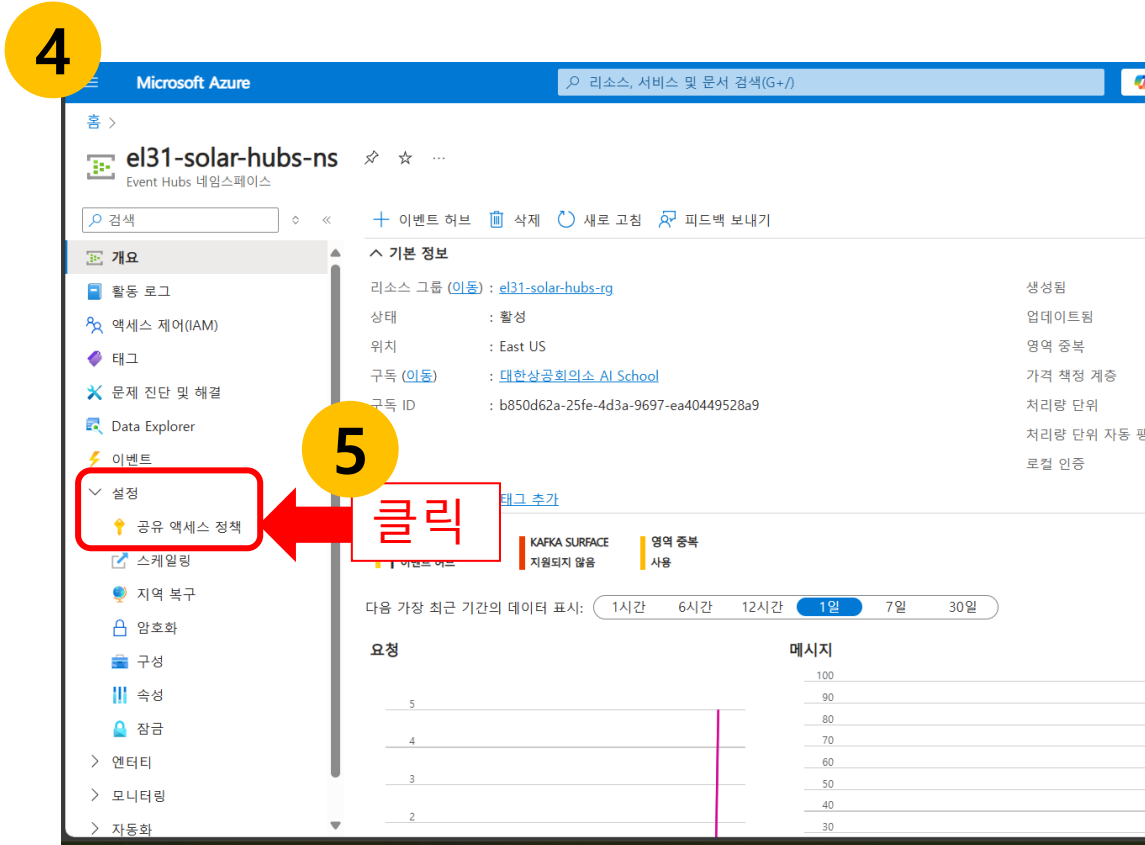
1. Azure Portal에 접속하여 로그인합니다. (<https://portal.azure.com>)
2. 앞서 생성한 eventhubs용 '리소스 그룹'을 선택합니다. (예: el31-solar-hubs-rg)
3. '리소스 그룹' 페이지에서 Event Hubs 네임스페이스를 선택합니다. (예: el31-solar-hubs-ns)

1



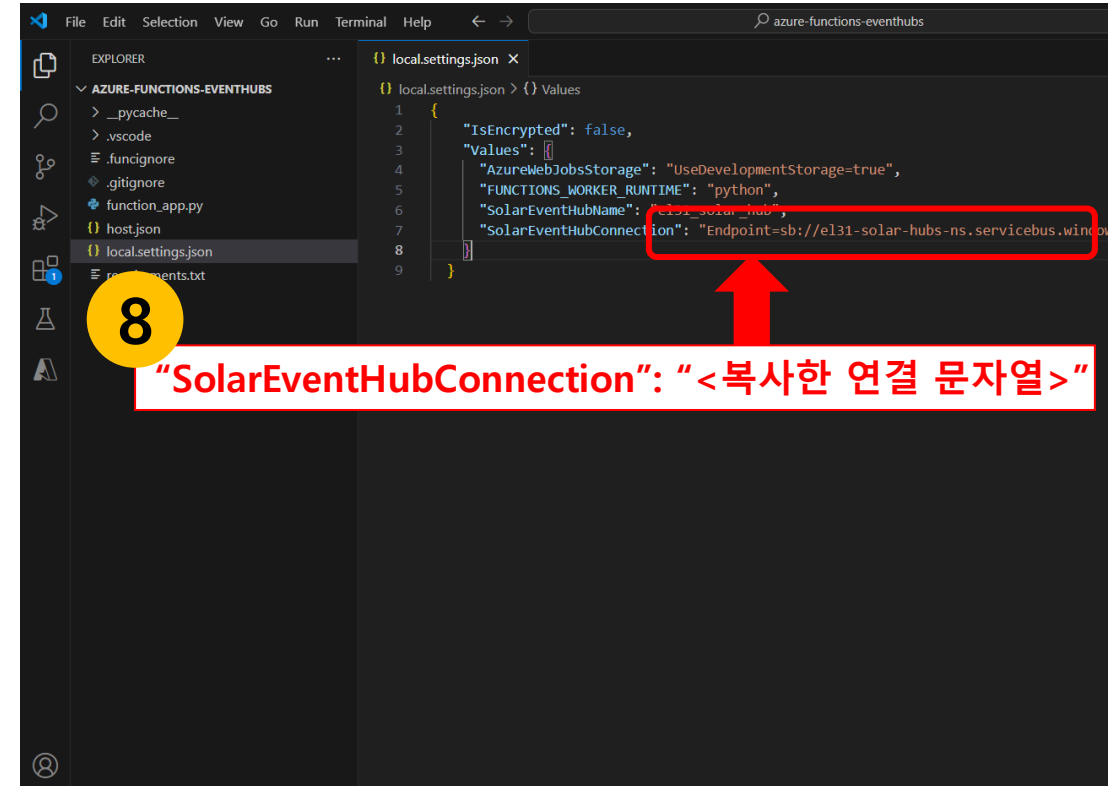
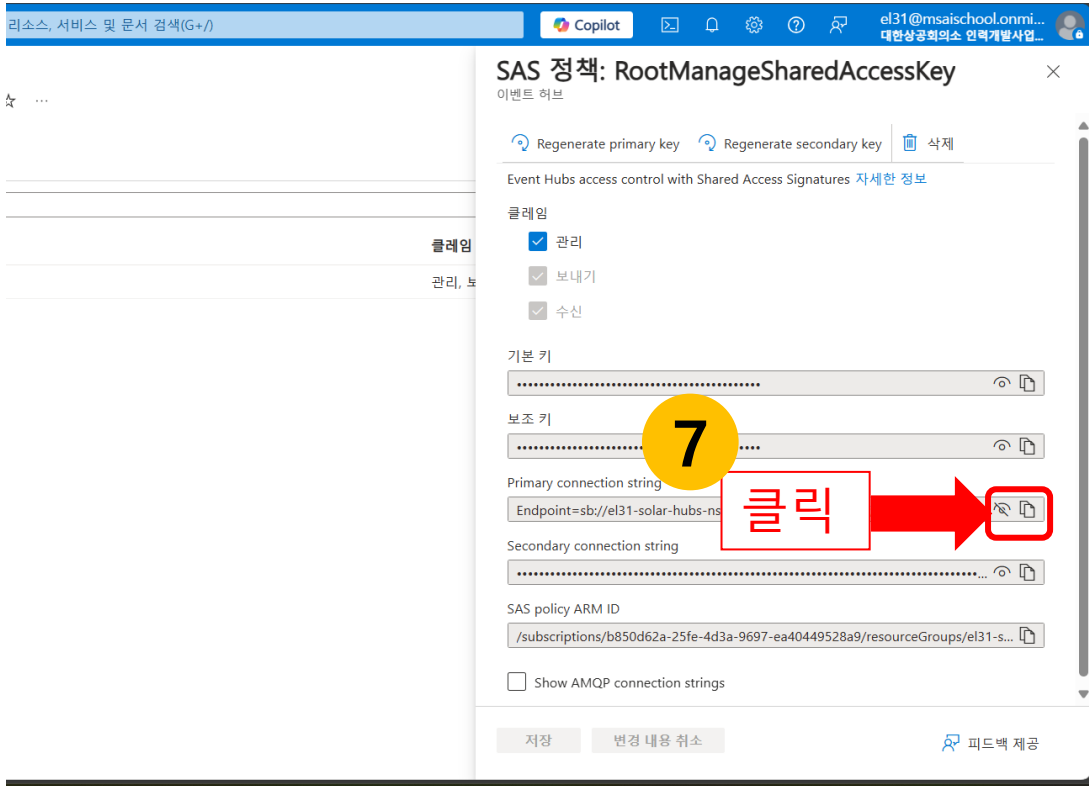
Event Hubs 연결 문자열 설정하기(2)

4. 'Event Hubs 네임스페이스' 페이지에 접속합니다.
5. '설정' 메뉴를 클릭 후 '공유 액세스 정책' 메뉴를 클릭합니다.
6. 'RootManageSharedAccessKey' 버튼을 클릭합니다.



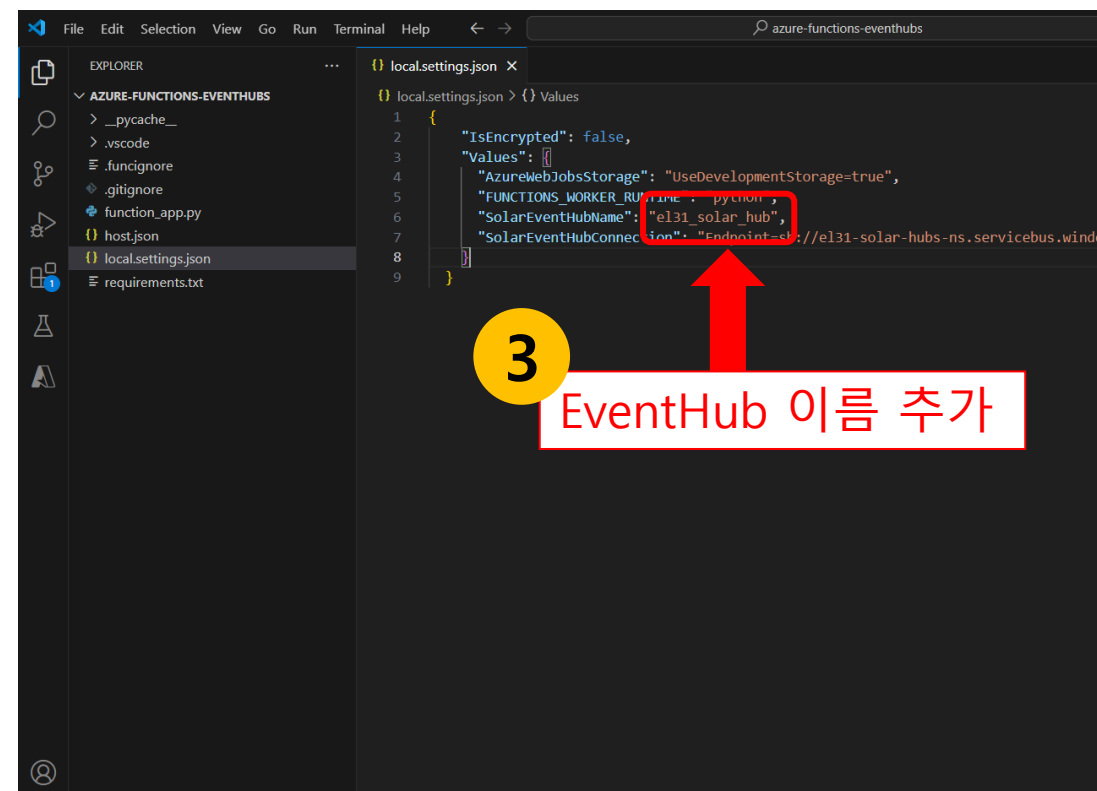
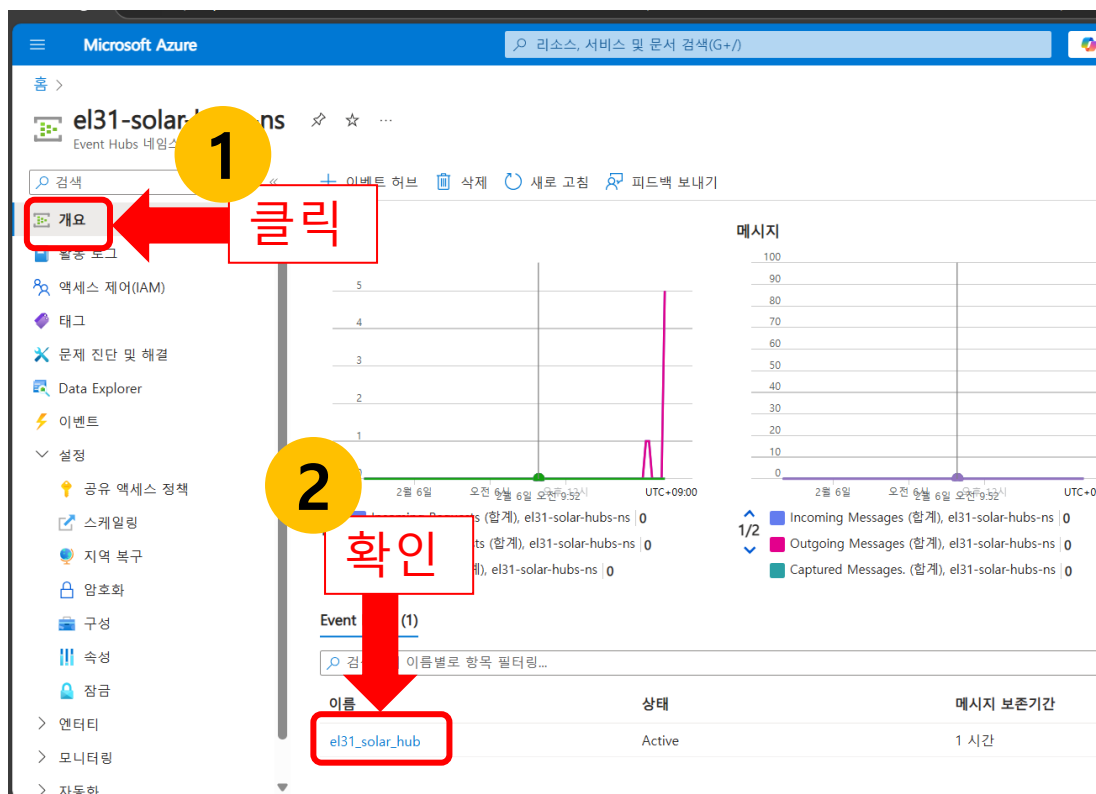
Event Hubs 연결 문자열 설정하기(3)

7. Primary connection string 항목 옆의 '복사' 버튼을 클릭하여 연결 문자열을 클립보드에 복사합니다.
8. 복사한 연결 문자열을 애플리케이션의 설정 파일에 추가해야 합니다.
local.settings.json 파일을 열고 다음과 같이 설정합니다. ("**SolarEventHubConnection**": "<복사한 연결 문자열>")



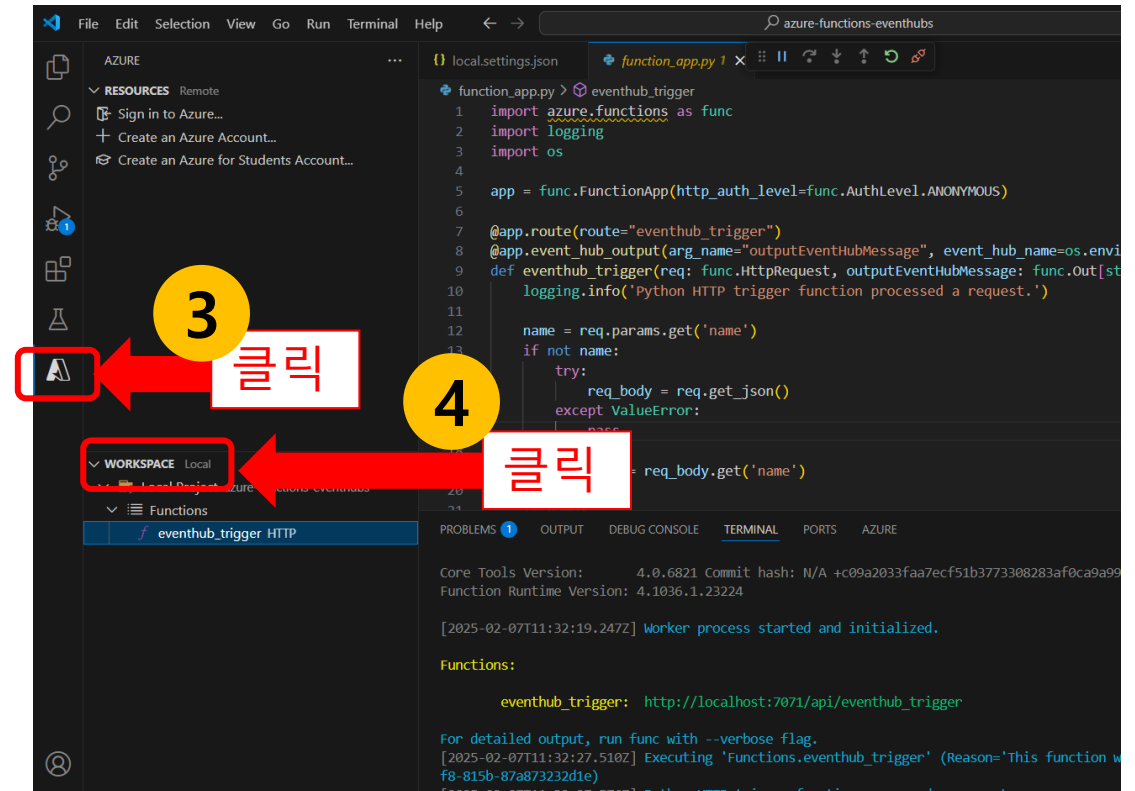
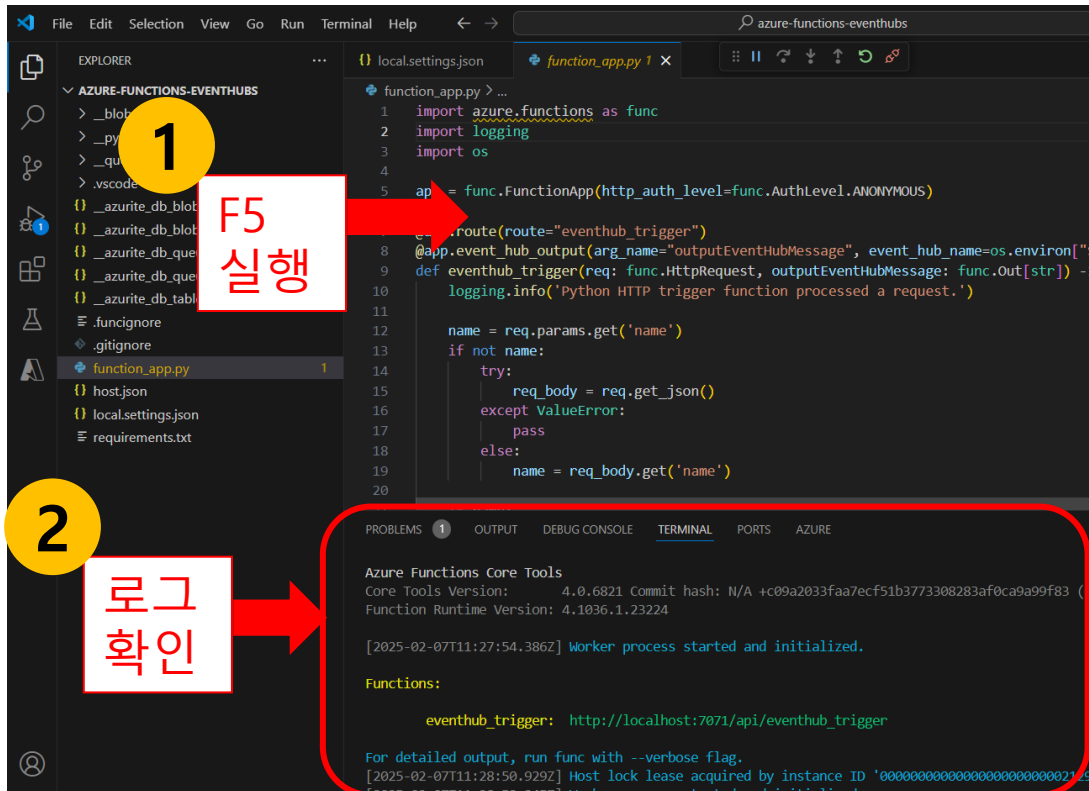
Azure Function에 Event Hub 지정하기

1. 'Event Hubs 네임스페이스 개요' 페이지에 접속합니다.
2. EventHub 이름을 확인합니다. (예: el31_solar_hub)
3. VS Code에서 "SolarEventHubName"에 EventHub 이름을 입력합니다.



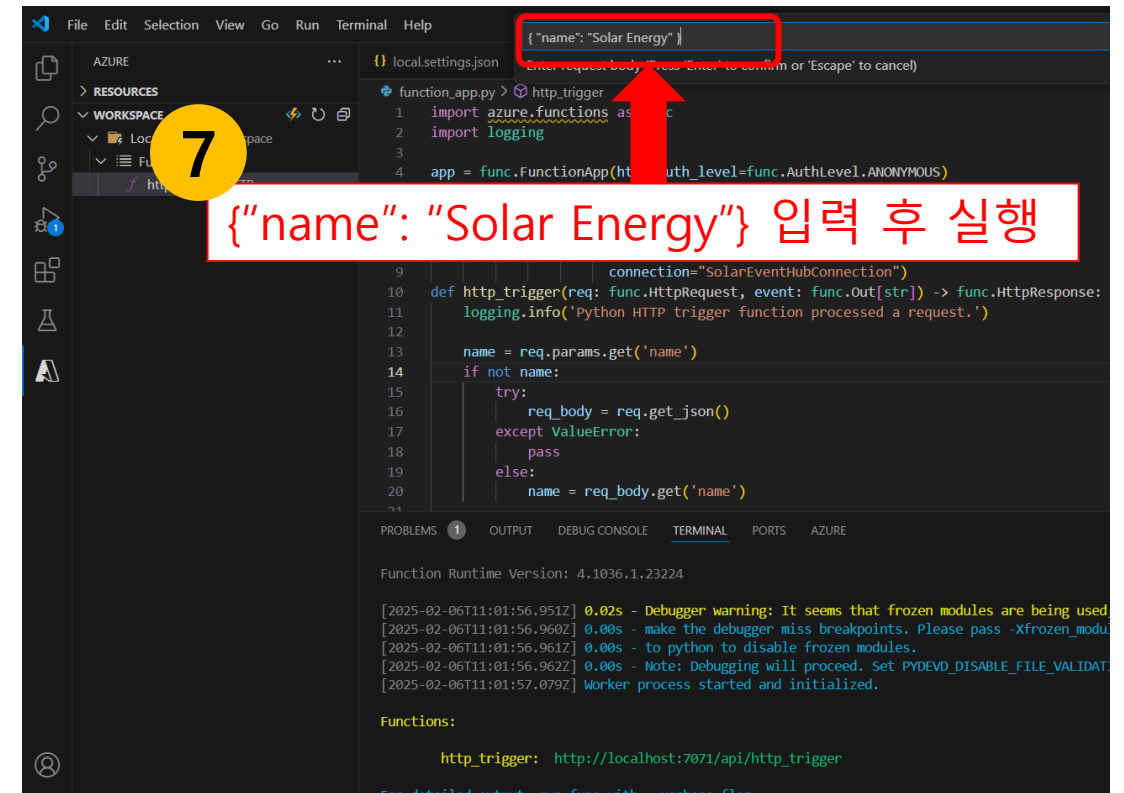
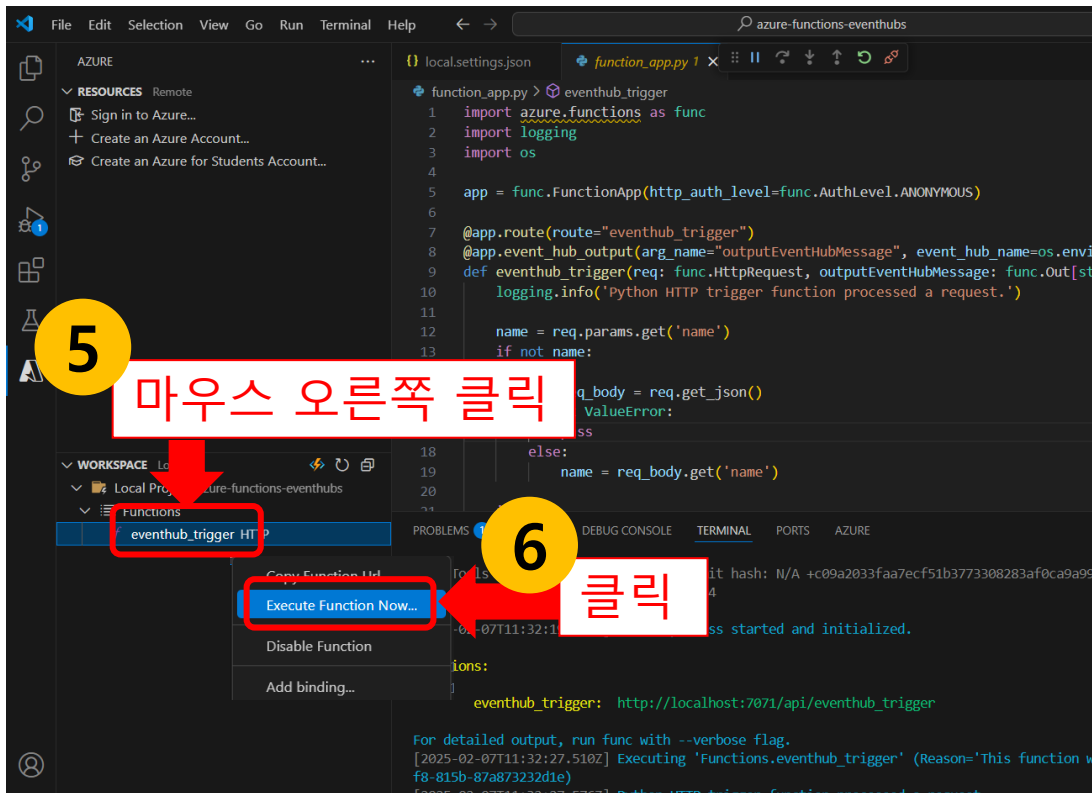
Azure Function 실행(1)

1. VS Code에서 'F5' 키를 입력하여 Azure Functions를 실행합니다.
2. '하단' 탭에 Azure Functions가 실행 된 것을 확인합니다.
3. '왼쪽' 패널에서 'Azure' 탭을 선택합니다.
4. 'WORKSPACE Local' 버튼을 클릭합니다.



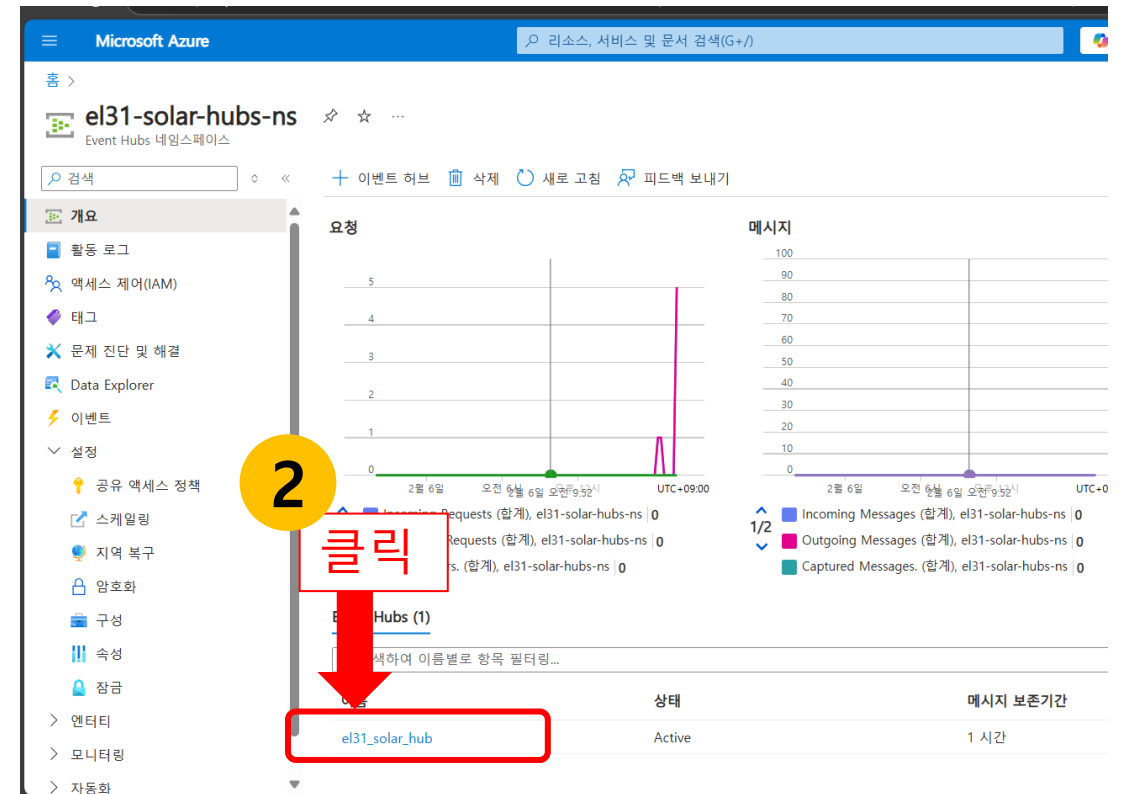
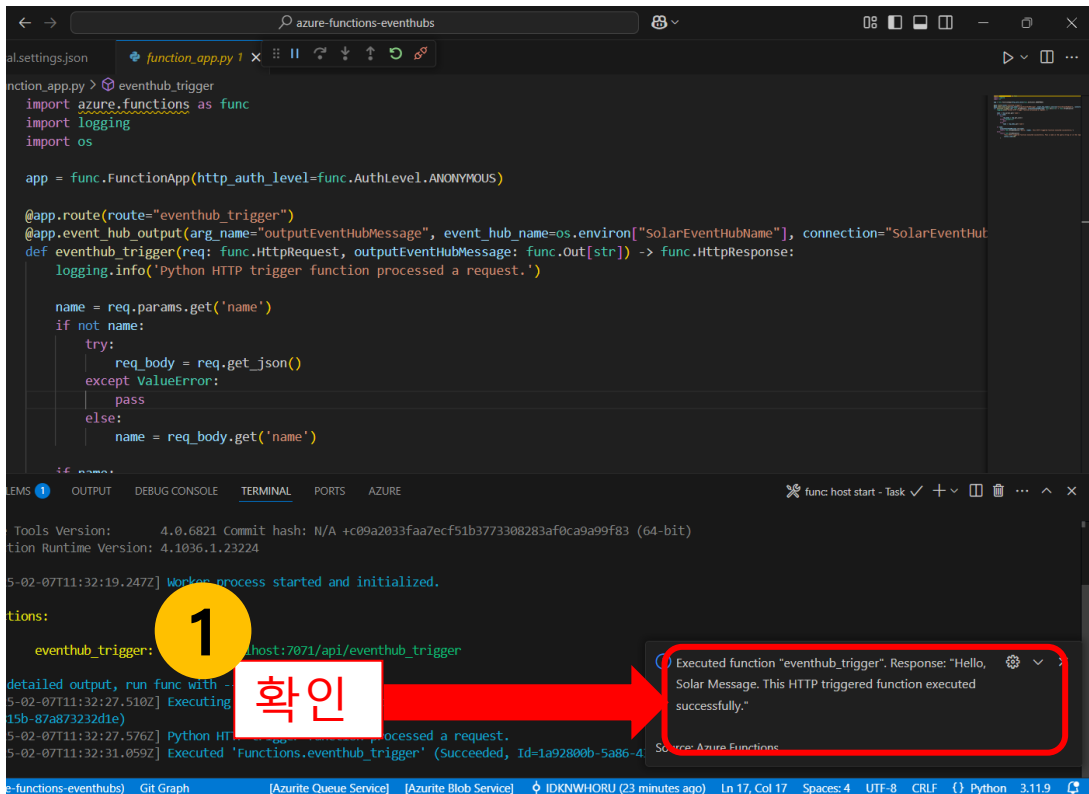
Azure Function 실행(2)

5. eventhub_trigger 함수에서 **마우스 오른쪽 클릭**을 합니다
6. **'Execute Function Now...'** 버튼을 클릭합니다
7. 요청 본문에 **{"name": "Solar Energy"}**를 입력 후 실행합니다



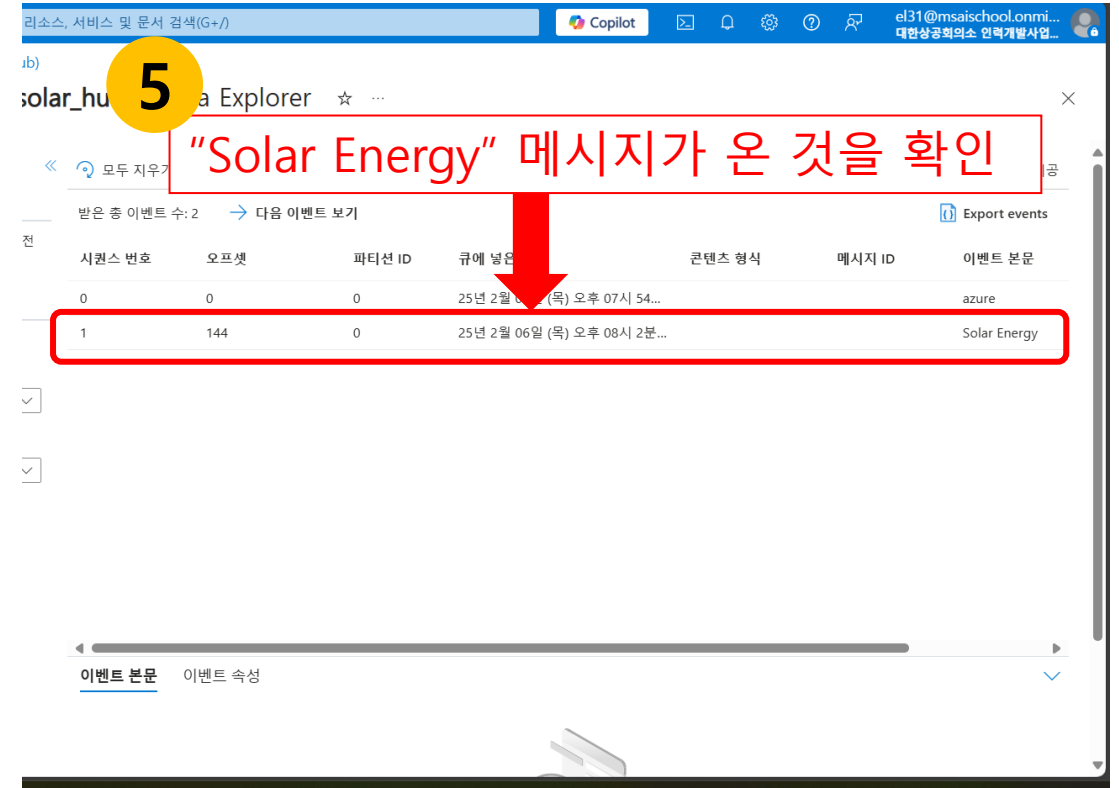
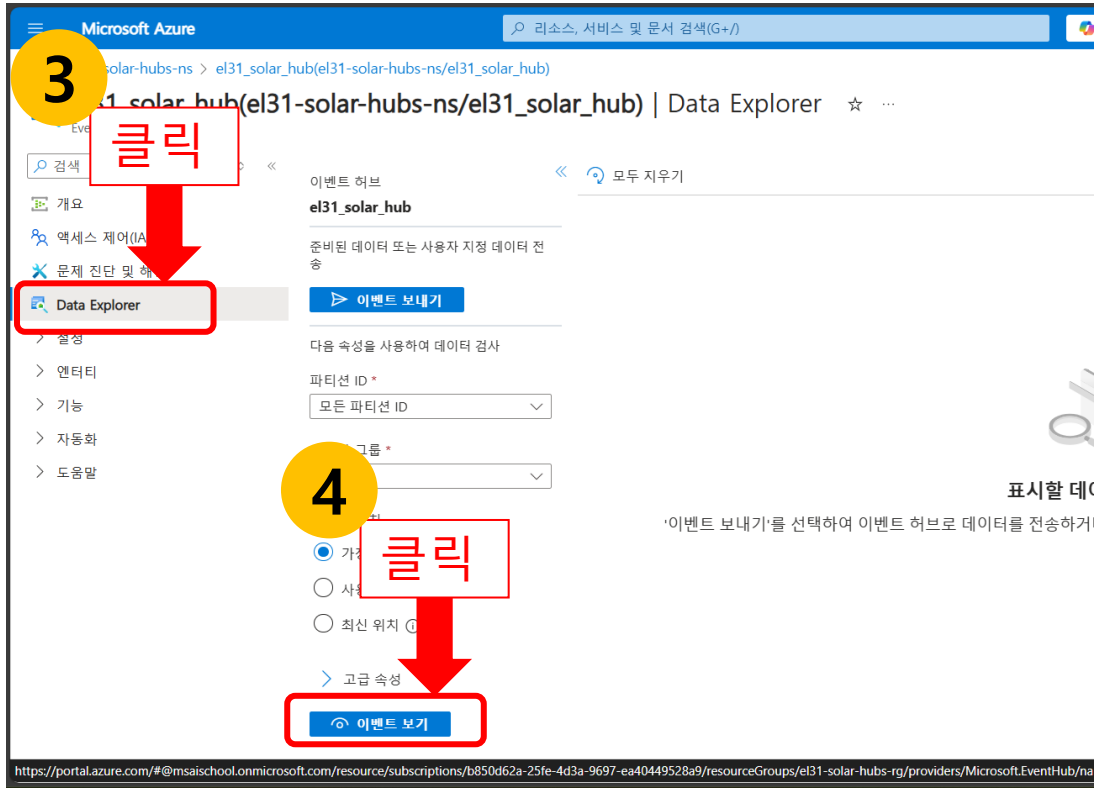
Event Hub 데이터 전송 확인하기(1)

1. Azure Functions를 실행하고 난 뒤 오른쪽 하단에 함수 실행 메시지를 확인합니다.
2. Azure Portal의 'EventHubs 네임스페이스 개요' 페이지에서 EventHub를 클릭합니다. (예: el31_solar_hub)



Event Hub 데이터 전송 확인하기(2)

3. 'Data Explorer' 메뉴를 선택합니다
4. '이벤트 보기' 버튼을 클릭합니다
5. 이벤트 본문에 Solar Energy 메시지가 온 것을 확인합니다



정리 요약

Azure Event Hubs는 실시간으로 대량의 데이터를 안정적이고 신뢰성 있게 수집, 처리 및 저장 할 수 있는 확장성 높은 데이터 스트리밍 플랫폼입니다.

- 다양한 목적으로 활용: 수집된 데이터를 이용해 실시간 분석, 대시보드 구축, 아카이빙, 기계 학습 등
- 세 가지 주요 구성 요소: 생산자, 이벤트 허브, 이벤트 소비자로 이루어져 있음
- 핵심 기능: Kafka 호환성과 스키마 레지스트리를 제공함
- 실전 사용 사례: 스마트 팩토리 환경에서 IoT 센서 데이터를 실시간으로 수집하고 분석
- 간단 실습: Azure Portal을 통해 Event Hub를 생성하고, Azure Functions와 연동하여 실시간 데이터 처리 파이프라인을 구축

감사합니다.

다음번 수업에서 만나요 😊